

(请大家预习)

第6章第3讲

人工神经网络

Artificial Neural Networks

张 燕 明

ymzhang@nlpr.ia.ac.cn

peopleucas.ac.cn/~ymzhang

模式分析与学习课题组(PAL)

多模态人工智能系统实验室 中科院自动化所

助教： 杨 奇 (yangqi2021@ia.ac.cn)

张 涛 (zhangtao2021@ia.ac.cn)

内容回顾

- 神经网络模型的监督训练
 - 准则函数的选择
 - 回归：均方误差(MSE)；分类：交叉熵(Cross Entropy)
 - 模型的选择
 - 隐层数、隐层节点数、激活函数
 - 算法：梯度下降(BP)
 - 权重初始化、学习率、冲量项(momentum)
 - 训练中的问题
 - 过拟合： $L2$ 正则化(regularization, weight decay)、早停(early stopping)
 - 梯度消失：激活函数、权重初始化、BN层
 - 局部极小：引入随机性

内容回顾

- 径向基函数(RBF)网络

$$z_j(\mathbf{x}) = \sum_{h=1}^H w_{hj} \phi_h(\mathbf{x}), \quad j = 1, 2, \dots, c$$

- 由多个径向基函数 $\{ \phi_h(\mathbf{x}) \}$ 线性组合而成
- 每个RBF $\phi_h(\mathbf{x})$ 包含一个中心 $\mathbf{w}_h$; $\mathbf{w}_h$ 与 $\mathbf{x}$ 的维数相同, 通常表示一个原型样本。
- $\phi_h(\mathbf{x})$ 是 $\| \mathbf{x} - \mathbf{w}_h \|$ 的单调减函数, 反映 $\mathbf{x}$ 与 $\mathbf{w}_h$ 的相似度

$$\text{例如: } \phi_h(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{w}_h\|^2}{2\sigma^2}\right)$$

- 确定一个RBF网络需要确定 $\{ w_{ih} \}$, $\{ w_{hj} \}$

内容回顾

- RBF vs MLP

RBF: $net_h = \|\mathbf{w}_h - \mathbf{x}\|^2, y_h = \exp(-\frac{net_h}{2\sigma^2}), z_j = \mathbf{w}_j^T \mathbf{x}$

MLP: $net_h = \mathbf{w}_h^T \mathbf{x}, y_h = \text{ReLU}(net_h), z_j = \mathbf{w}_j^T \mathbf{x}$

与MLP相似，RBF网络是全连接、包含一个隐层的前馈网络；其特殊之处在于隐层激活函数和净输入计算方式。

内容回顾

- 介绍

- 神经网络、深度学习的发展历史
- 基本拓扑结构：前馈/反馈，全连接/局部连接
- 学习算法：有监督(BP)/无监督(Hebb's rule, 竞争学习)

- 基本模型

- 神经元模型、单层感知器、多层感知器

- 扩展模型

- RBF网络、Hopfield网络、玻尔兹曼机、受限玻尔兹曼机、自组织映射网络
- **Deep neural networks:** CNN, Autoencoder, RNN, LSTM, Transformer, GNN, etc.

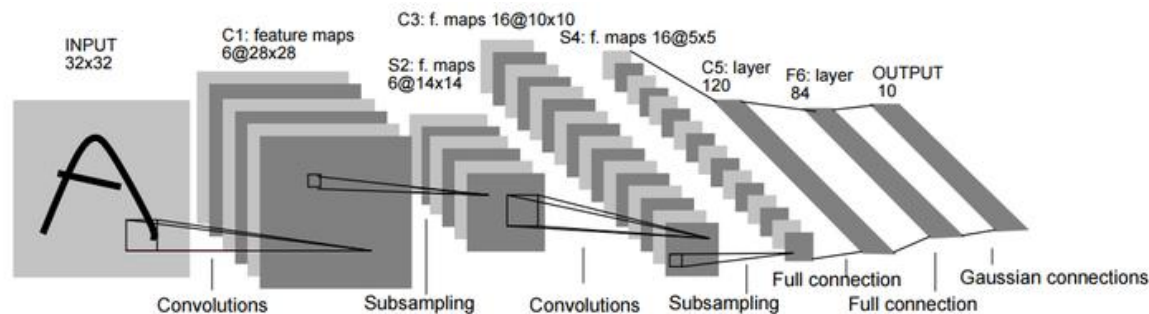
6.9.3 卷积神经网络

- **Convolutional Neural Network (CNN)**
 - CNN是图像识别领域的经典方法。
 - CNN的显著特点是**局部连接**和**权值共享**，其网络结构更加类似于生物神经网络。由于权值的数量的减少，网络模型的复杂度更低。
 - CNN本质上是一个多层感知器，这种网络结构对平移、比例缩放、倾斜或者其它形式的变形具有一定的不变性。

6.9.3 卷积神经网络

- CNN

- **1962年**，Hubel和Wiesel通过对猫视觉皮层细胞的研究（局部敏感和方向选择的神经元具有独特的网络结构），提出了感受野 (receptive field) 的概念。
- **1984年**，日本学者 Kunihiro Fukushima 提出了神经认知机 (**neocognitron**)，其中包含了卷积、下采样等CNN的核心操作，由此创造了CNN的基本架构；但neocognitron采用无监督方式训练。
- **1989~1998年**，Yann LeCun等将BP算法应用到CNN的有监督训练，并设计了第一个实用的CNN框架 **LeNet-5**，用于手写数字识别；LeNet-5是现代计算机视觉CNN的原型。



6.9.3 卷积神经网络

- 卷积：信号处理的基本操作



卷积操作的要素

0	2	3	3	5	6	1	2	3
0	1	4	3	5	1	1	0	5
3	2	0	1	4	5	2	1	6
4	1	1	2	0	1	3	0	3
0	1	3	2	1	1	1	2	2
1	0	2	0	2	2	0	2	0
1	2	0	5	0	2	1	4	0
0	3	2	4	1	0	4	3	0
1	1	1	2	3	0	3	0	0

输入图像
(灰度图)

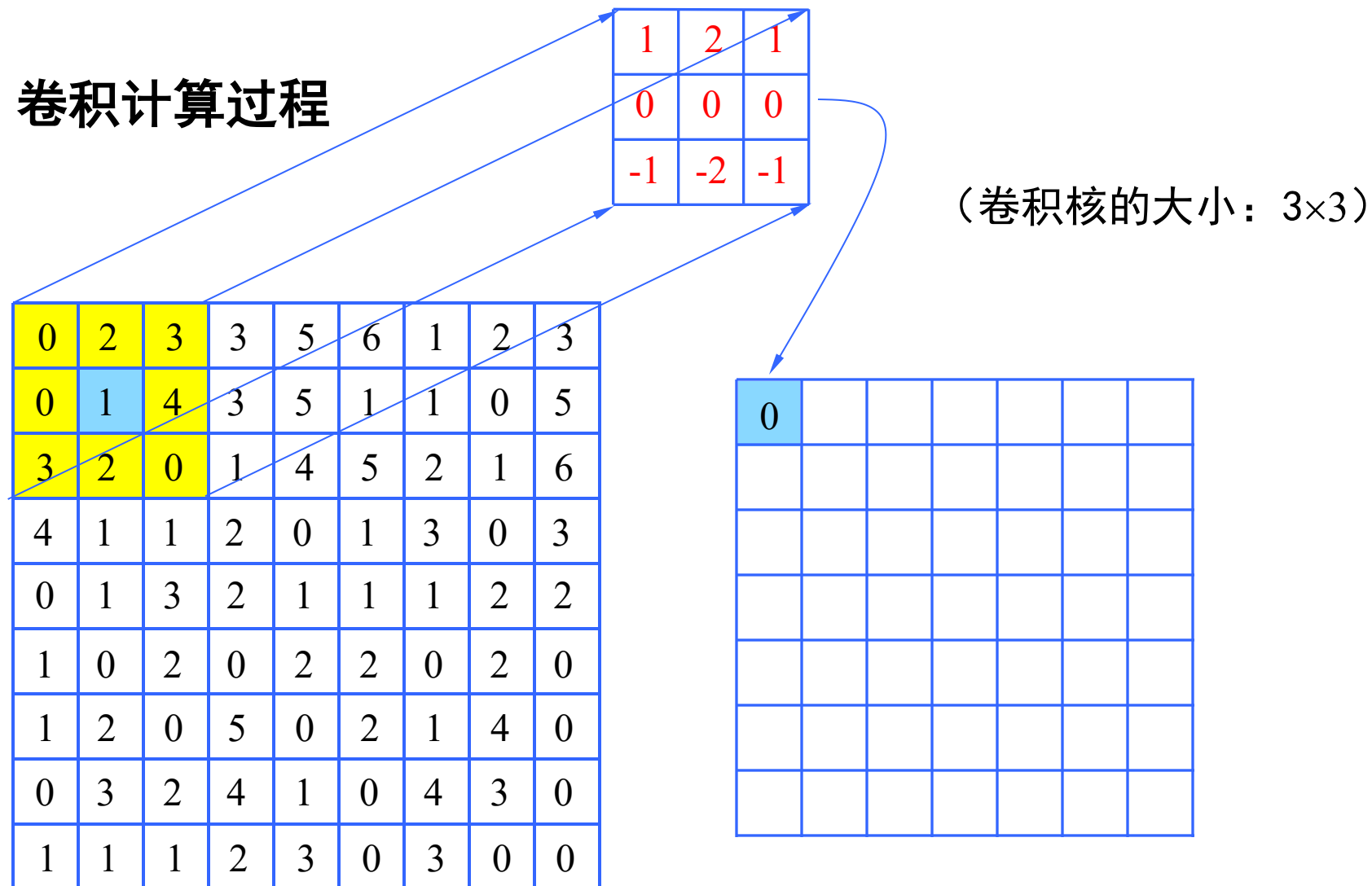
1	2	1
0	0	0
-1	-2	-1

卷积核
(卷积核的大小: 3×3)

输出特征图

一个包含可调参数的小窗口

卷积计算过程



$$0 \times 1 + 2 \times 2 + 3 \times 1 + 0 \times 0 + 1 \times 0 + 4 \times 0 + 3 \times (-1) + 2 \times (-2) + 0 \times (-1) = 0$$

卷积计算过程

1	2	1
0	0	0
-1	-2	-1

(卷积核的大小: 3×3)

0	2	3	3	5	6	1	2	3
0	1	4	3	5	1	1	0	5
3	2	0	1	4	5	2	1	6
4	1	1	2	0	1	3	0	3
0	1	3	2	1	1	1	2	2
1	0	2	0	2	2	0	2	0
1	2	0	5	0	2	1	4	0
0	3	2	4	1	0	4	3	0
1	1	1	2	3	0	3	0	0

0	8					

$$2 \times 1 + 3 \times 2 + 3 \times 1 + 1 \times 0 + 4 \times 0 + 3 \times 0 + 2 \times (-1) + 0 \times (-2) + 1 \times (-1) = 8$$

卷积计算过程

1	2	1
0	0	0
-1	-2	-1

(卷积核的大小: 3×3)

0	2	3	3	5	6	1	2	3
0	1	4	3	5	1	1	0	5
3	2	0	1	4	5	2	1	6
4	1	1	2	0	1	3	0	3
0	1	3	2	1	1	1	2	2
1	0	2	0	2	2	0	2	0
1	2	0	5	0	2	1	4	0
0	3	2	4	1	0	4	3	0
1	1	1	2	3	0	3	0	0

0	8	8				

$$3 \times 1 + 3 \times 2 + 5 \times 1 + 4 \times 0 + 3 \times 0 + 5 \times 0 + 0 \times (-1) + 1 \times (-2) + 4 \times (-1) = 8$$

卷积计算过程

1	2	1
0	0	0
-1	-2	-1

(卷积核的大小: 3×3)

0	2	3	3	5	6	1	2	3
0	1	4	3	5	1	1	0	5
3	2	0	1	4	5	2	1	6
4	1	1	2	0	1	3	0	3
0	1	3	2	1	1	1	2	2
1	0	2	0	2	2	0	2	0
1	2	0	5	0	2	1	4	0
0	3	2	4	1	0	4	3	0
1	1	1	2	3	0	3	0	0

0	8	8				
						6

$$1 \times 1 + 4 \times 2 + 0 \times 1 + 4 \times 0 + 3 \times 0 + 0 \times 0 + 3 \times (-1) + 0 \times (-2) + 0 \times (-1) = 6$$

卷积计算过程

1	2	1
0	0	0
-1	-2	-1

(卷积核的大小: 3×3)

	0	2	3	3	5	6	1	2	3
	0	1	4	3	5	1	1	0	5
	3	2	0	1	4	5	2	1	6
	4	1	1	2	0	1	3	0	3
	0	1	3	2	1	1	1	2	2
	1	0	2	0	2	2	0	2	0
	1	2	0	5	0	2	1	4	0
	0	3	2	4	1	0	4	3	0
	1	1	1	2	3	0	3	0	0

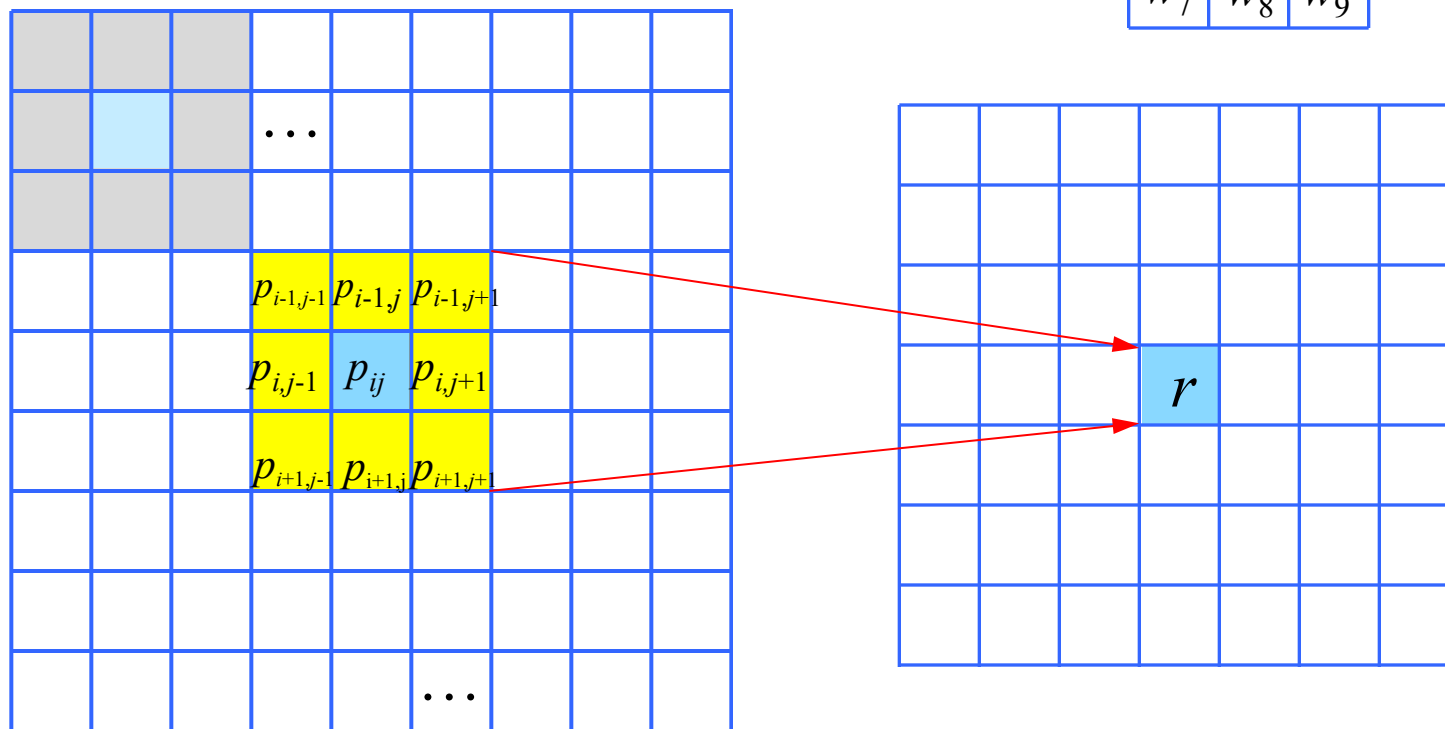
为了在每个输入位置上计算卷积，需要边缘补齐

use Padding!

(卷积核的大小: 3×3)

二维卷积的一般化描述

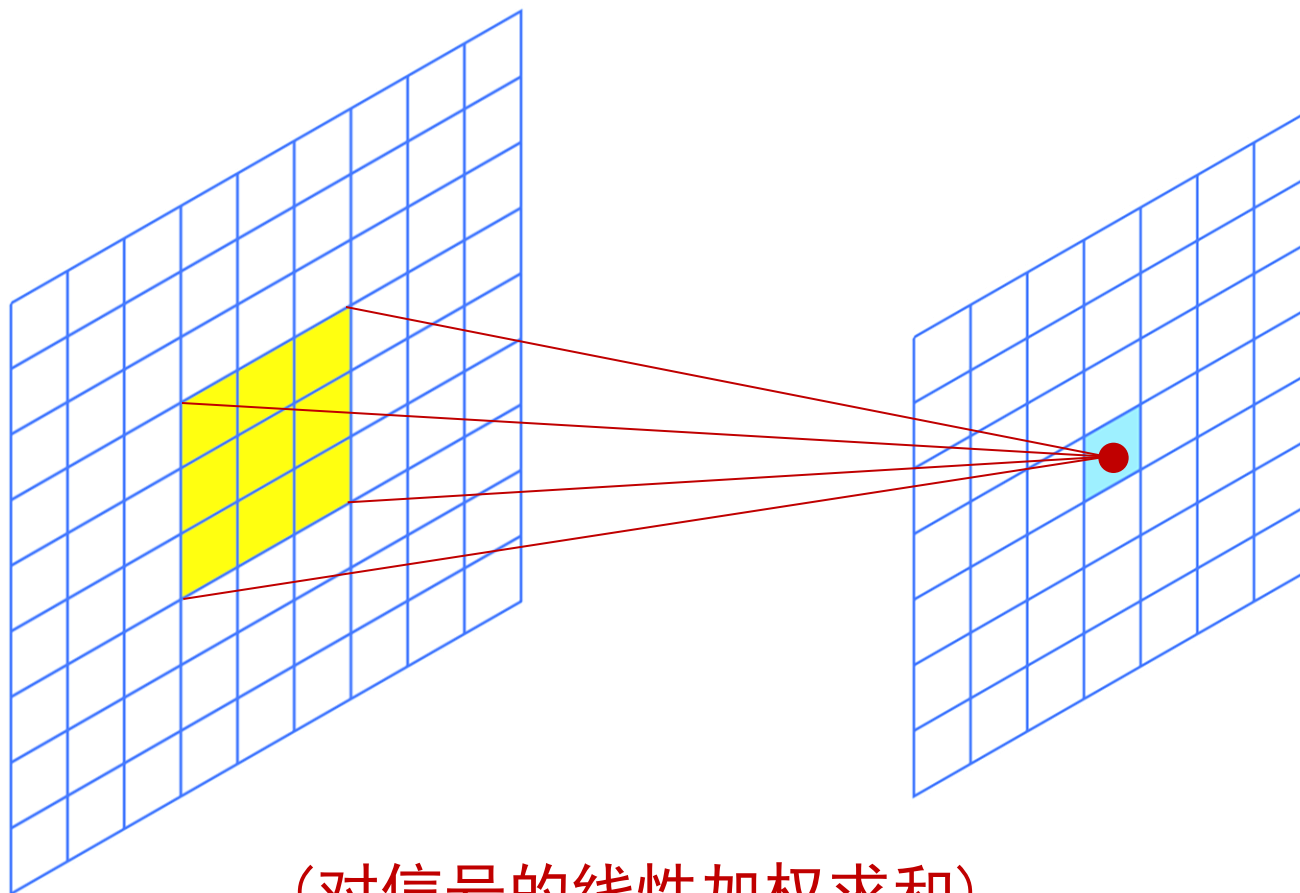
w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9



$$\begin{aligned} r = & p_{i-1,j-1}w_1 + p_{i-1,j}w_2 + p_{i-1,j+1}w_3 + p_{i,j-1}w_4 + p_{i,j}w_5 + p_{i,j+1}w_6 \\ & + p_{i+1,j-1}w_7 + p_{i+1,j}w_8 + p_{i+1,j+1}w_9 \end{aligned}$$

(对信号的线性加权求和)

二维卷积的一般化描述



(对信号的线性加权求和)

扩大卷积核的大小

感受野(receptive field)

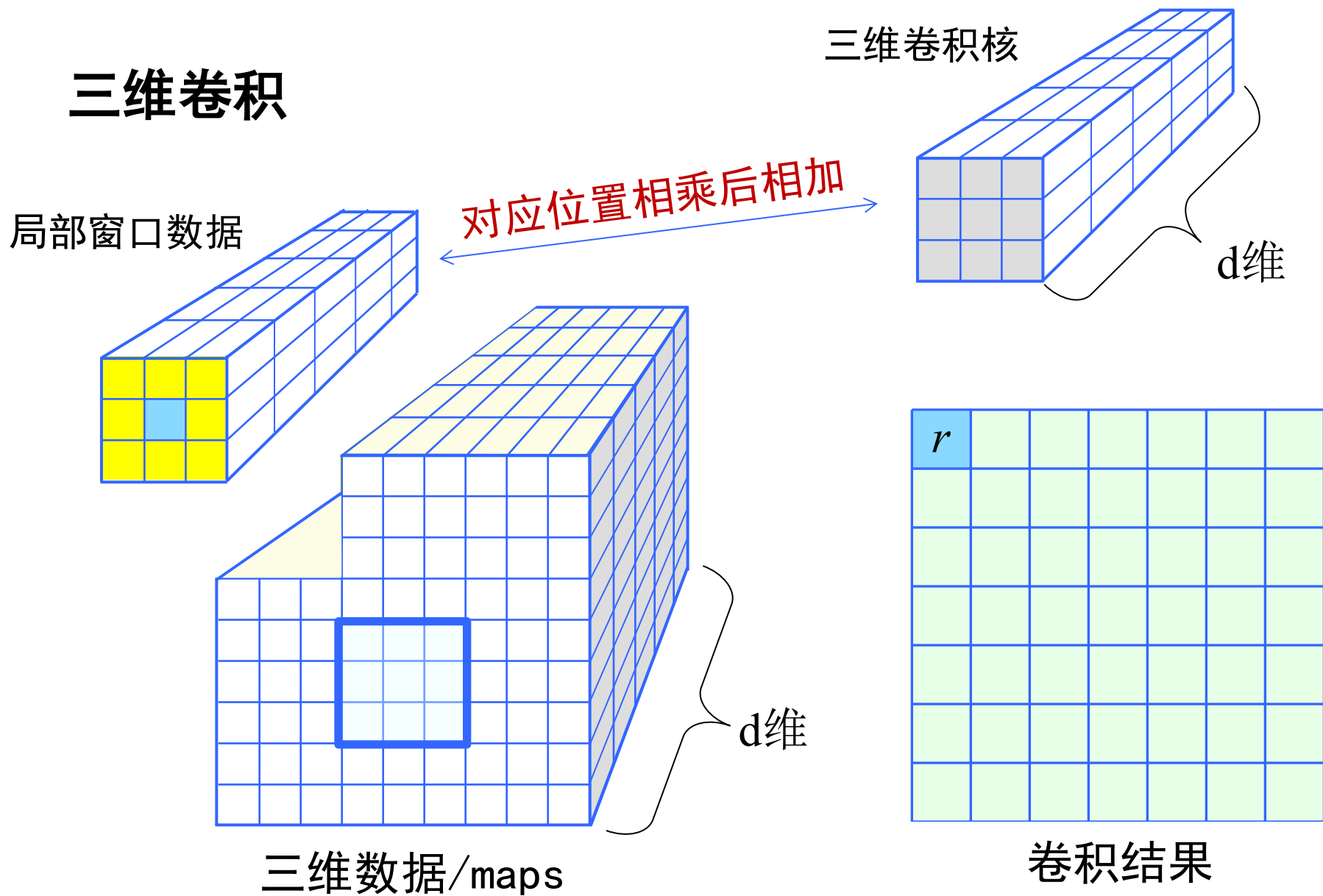
(卷积核的大小: 5×5)

w_1	w_2	w_3		
		w_{23}	w_{24}	w_{25}

p_1	p_2	p_3						p_9
			...					
p_{73}					...	p_{79}	p_{80}	p_{81}

(对信号的线性加权求和)

三维卷积



图像卷积操作一举例

$$\frac{1}{9} \times \begin{array}{|c|c|c|} \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline \end{array}$$



(平均平滑)

图像卷积操作一举例

1	1	1
1	-8	1
1	1	1



(拉普拉斯边缘检测)

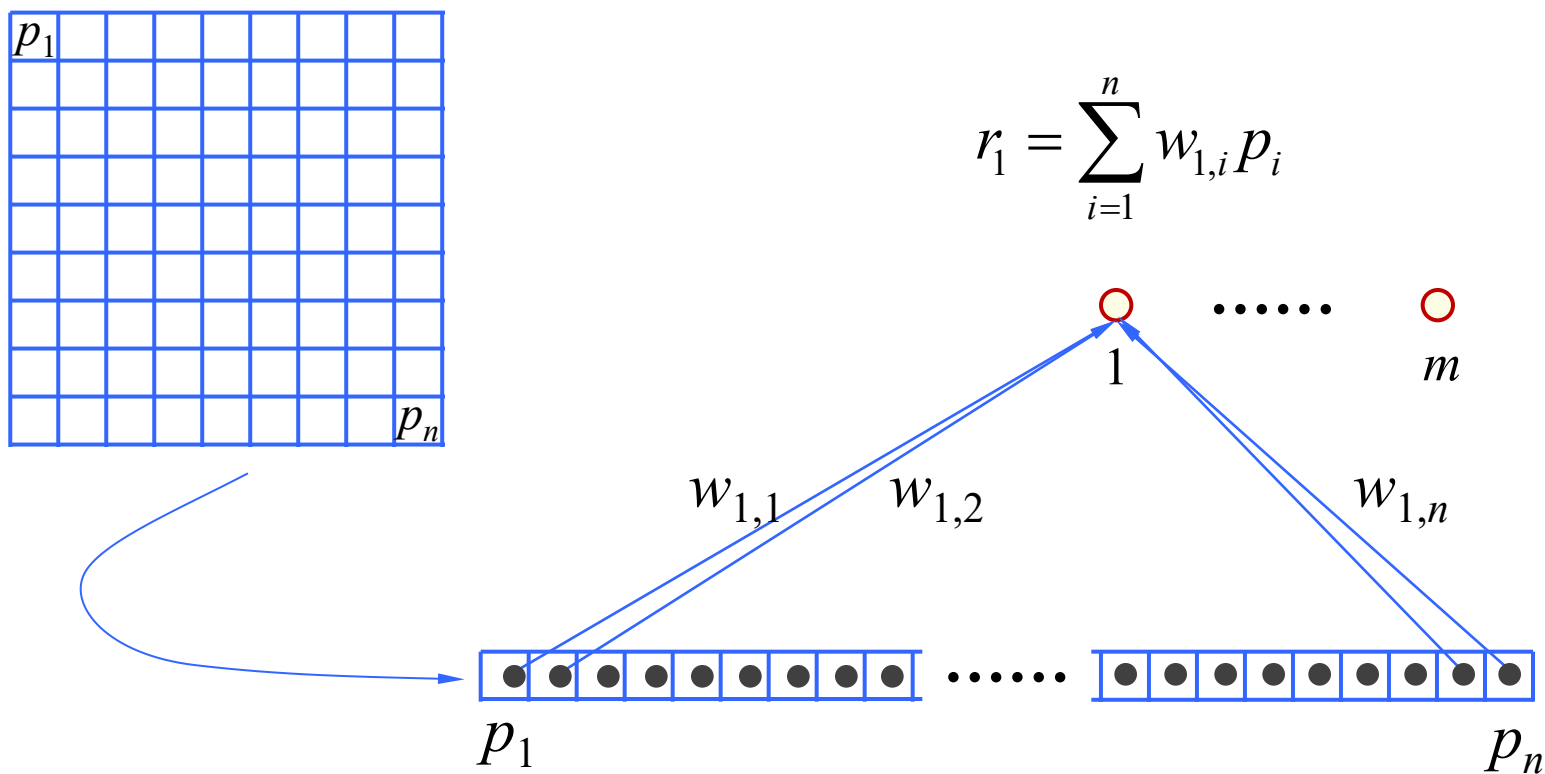
6.9.3 卷积神经网络

- 卷积操作小结

- 在空间上，卷积是一种**局部窗口内**的运算，即将卷积核覆盖在该窗口上。
- 卷积操作是线性的，是**线性加权求和**，结果被存放于窗口中心的像素上。
- 卷积核记录了一组权值，在从图像左上角到右下角的滑动覆盖的过程中**保持不变**。
- 每个卷积核相当于对某一种特征的提取器、检测器。

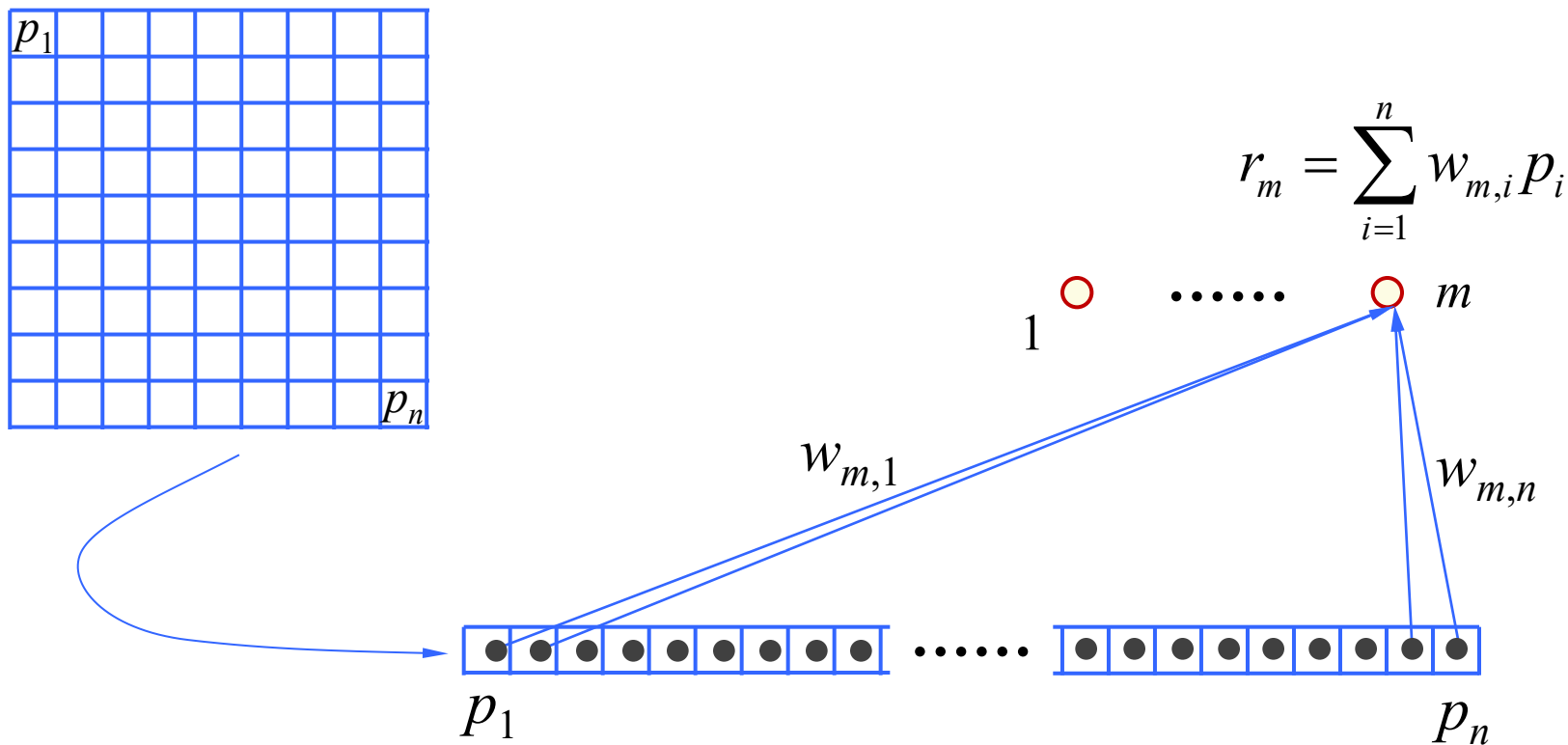
6.9.3 卷积神经网络

- 从加权求和的角度——按全连接



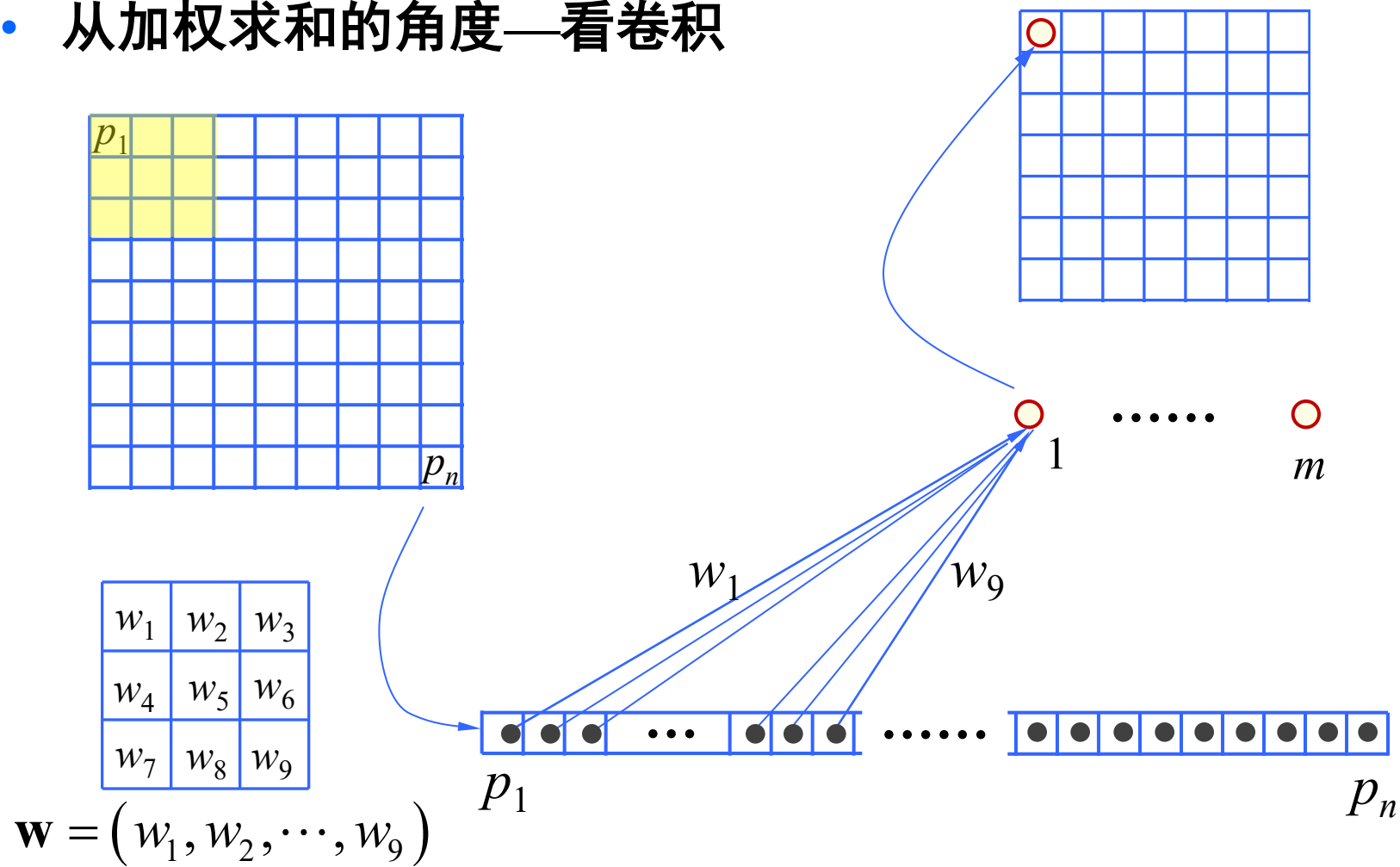
6.9.3 卷积神经网络

- 从加权求和的角度——按全连接



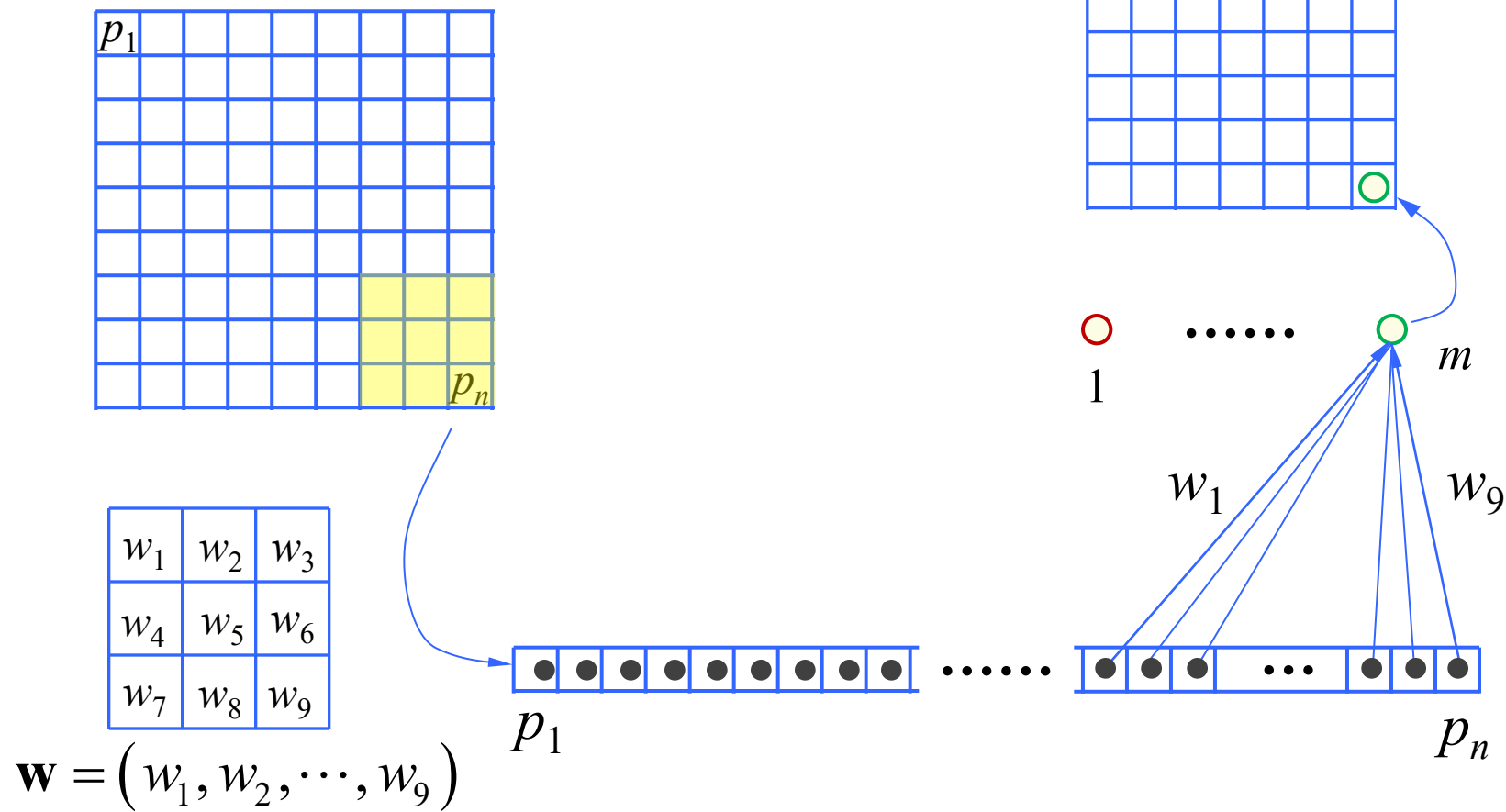
6.9.3 卷积神经网络

- 从加权求和的角度—看卷积



6.9.3 卷积神经网络

- 从加权求和的角度—看卷积



6.9.3 卷积神经网络

- 全连接的问题

- 如果以图像作为输入，并将每个像素看成一个输入层结点，对于200×200大小的图像，则输入层就有4万个结点；
- 如果第一隐含层仅仅包含1000个结点，则权重数量将达到：

$$4万 \times 1000 = 4千万$$

- 这是巨大的计算负担和学习负担。

6.9.3 卷积神经网络

- 降低网络权重数量—**局部连接**

- 视觉系统局部感受野

- 视觉生理学相关研究普遍认为：人对外界的认知是从局部到全局的。视觉皮层的神经元就是局部接受信息的（即只响应某些特定区域的刺激）

- **图像空间相关性**：对图像而言，局部邻域内的像素联系较紧密，距离较远的像素相关性则较弱。

- **神经网络由全连接变为部分连接**

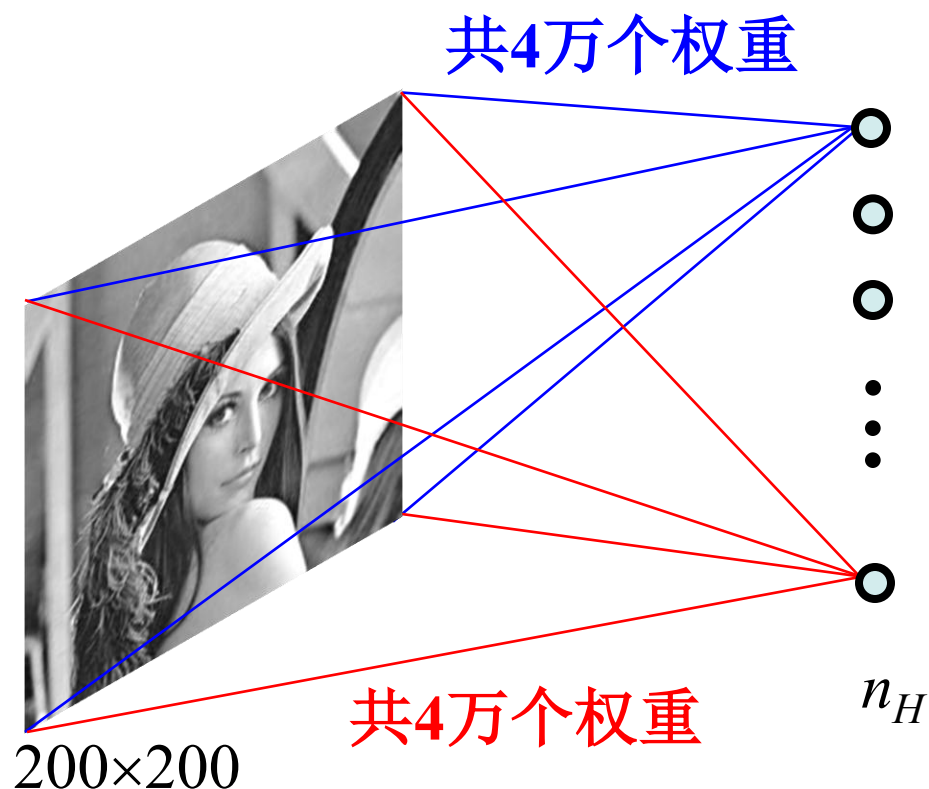
- 每个神经元其实没有必要对全局图像进行感知，只需要对局部进行感知。（局部感受野）
- 在更高层将局部的信息综合起来获得全局信息。

6.9.3 卷积神经网络

- 降低网络权重数量—**权值共享**
 - 局部连接可降低网络的权重数量，但仍不足够。
 - 引入**权值共享机制**。这一机制是：“从图像任何一个局部区域内连接到同一类型的隐含结点，其权重保持不变”。
 - **权值共享保证了CNN提取的特征具有平移不变性**

6.9.3 卷积神经网络

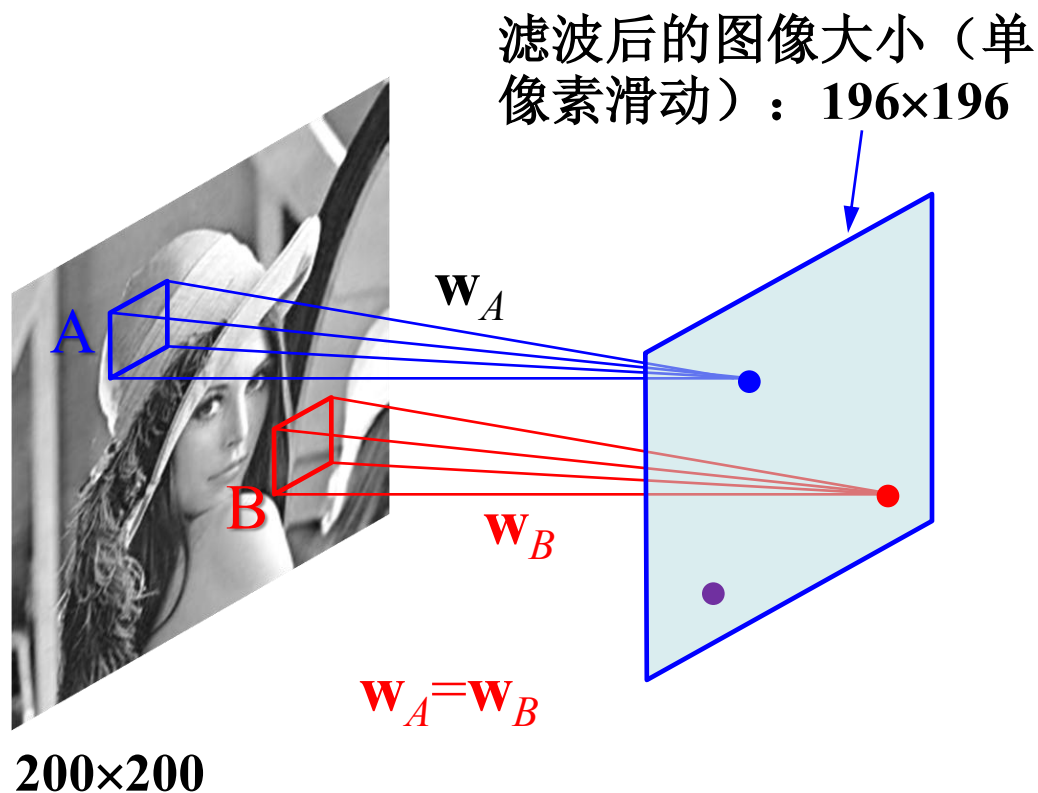
- 全连接



(如果 $n_H=4000$ ，则共有1亿6千万个权重需要学习)

- 局部连接

- 考虑 5×5 大小的卷积核且权值共享



若采用全连接，权重数为：

$$200\times 200 \times 196\times 196$$



若采用局部连接，权重数为：

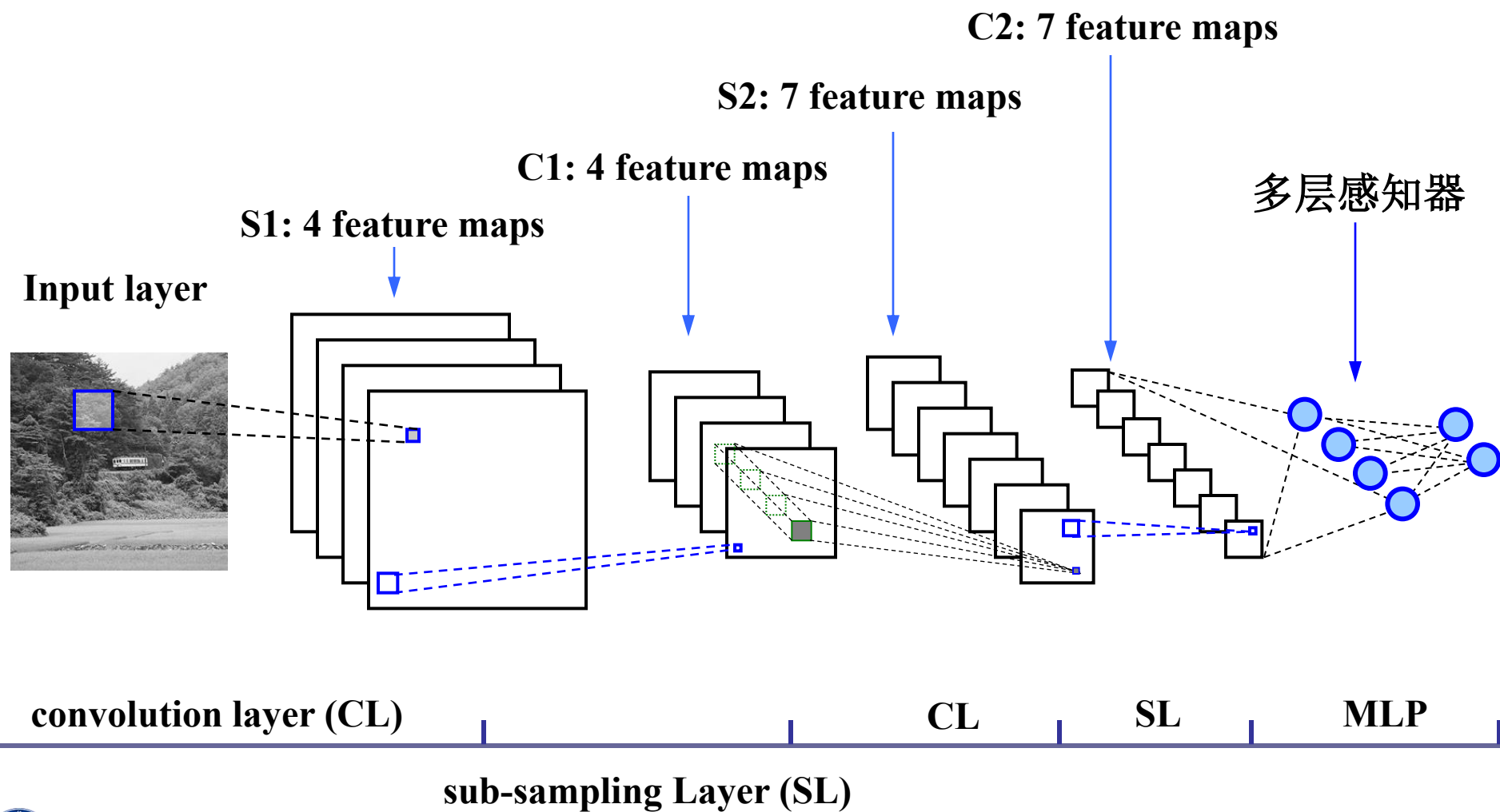
$$25 \times 196\times 196$$



若采用局部连接+权值共享：一共25个权重！

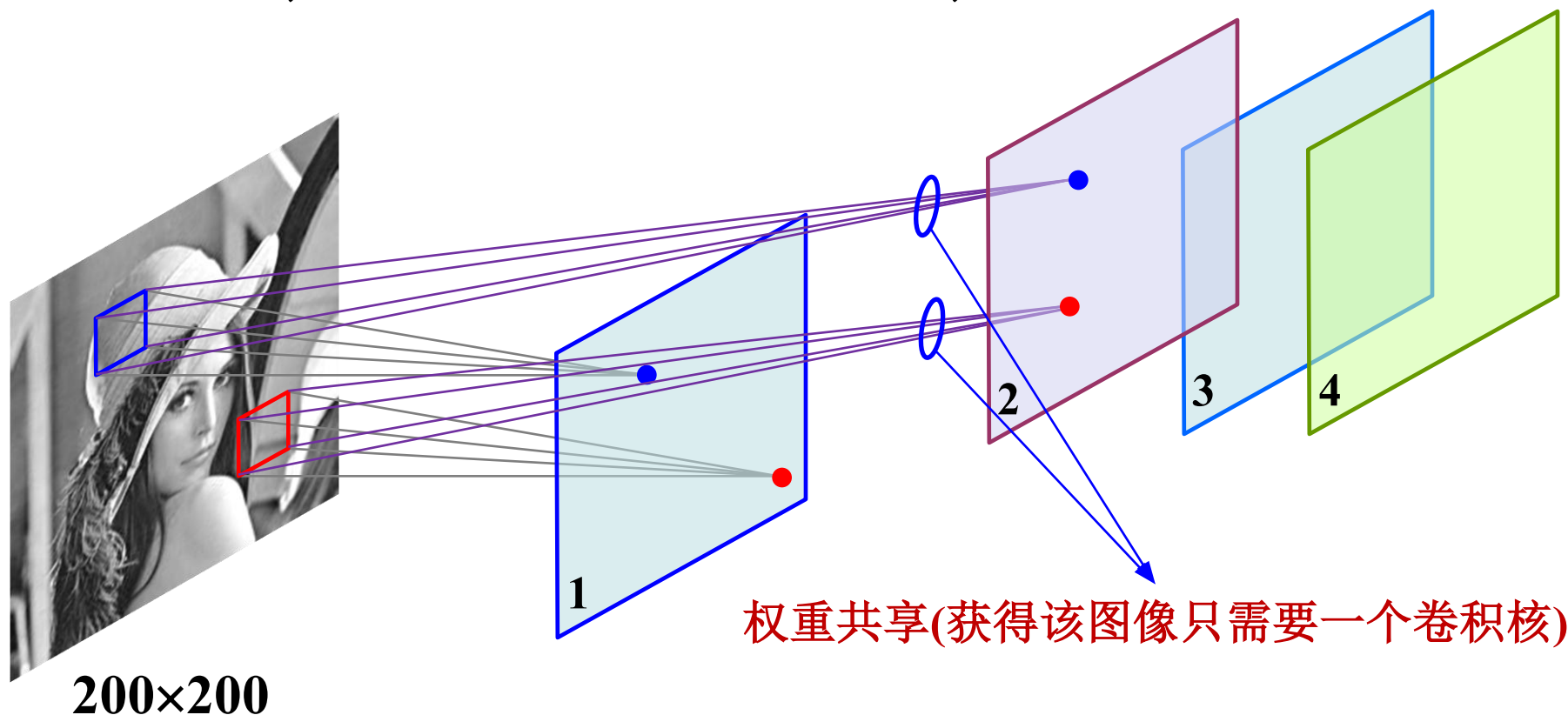
6.9.3 卷积神经网络

- 例子：设计一个用于图像分类的CNN网络



6.9.3 卷积神经网络

- 第一个隐含层
 - 每个卷积核相当于对某一种特征的提取器、检测器
 - 因此，实际中需要使用多个卷积核，比如4个



从输入层至第一隐层权重仅有：25×4个！

6.9.3 卷积神经网络

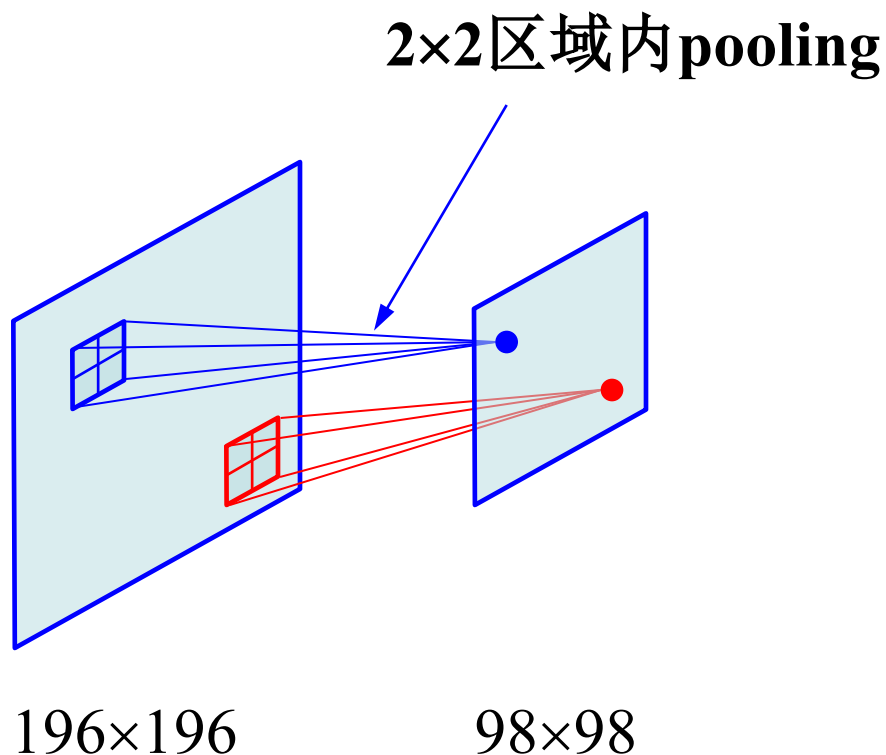
- 第一个隐含层
 - 考虑4个 5×5 卷积核的情形
 - 每个卷积核通过卷积操作获得的特征图像的大小为 196×196
 - 如果采用这些特征设计分类器，数据空间的维数将达到 $4 \times 196 \times 196$ 。实际中卷积核更多，可能导致一个高维问题。采用高维数据设计分类器，容易产生过拟合。
 - 一个办法就是对特征图像下采样：聚合统计/池化(pooling)，比如，计算图像一个区域上的某个特定特征的平均值(或最大值)

6.9.3 卷积神经网络

- 图像区域内关于某个特征的统计聚合
 - 两种方式：区域内取最大值或平均



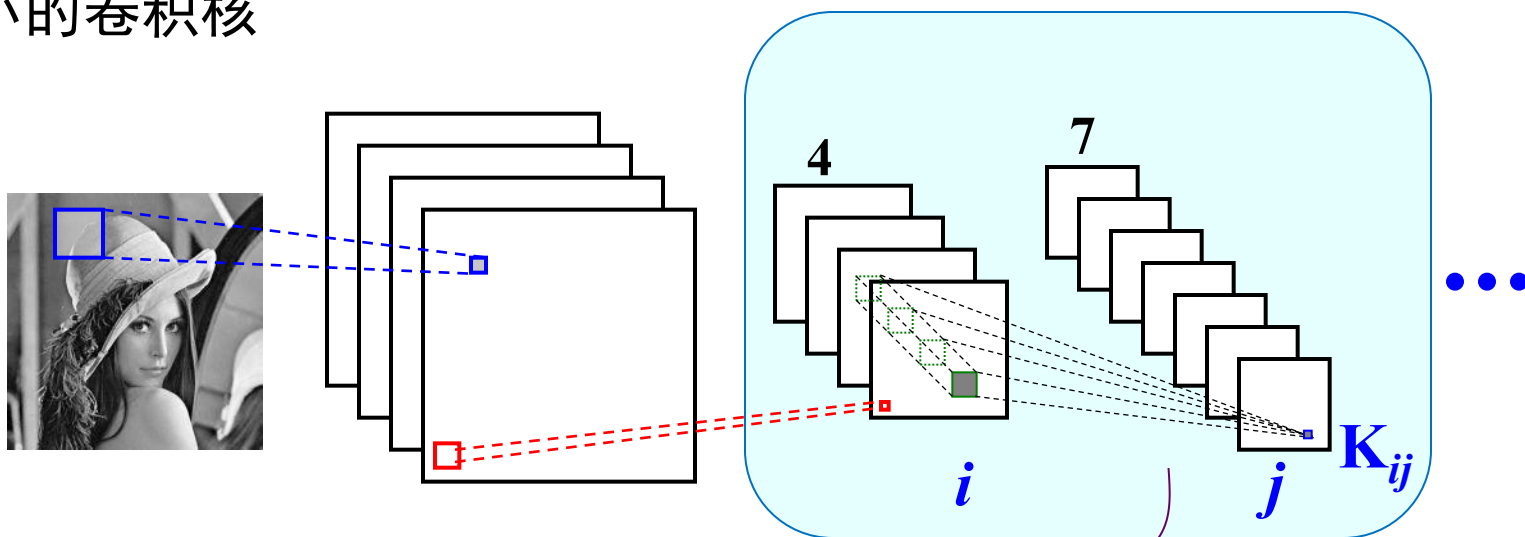
200×200



6.9.3 卷积神经网络

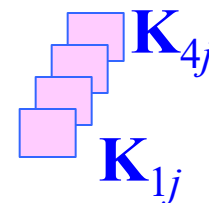
- 目前完成的步骤
 - 对图像，采用多个卷积核进行卷积，获得多个滤波结果，每一个滤波结果对应一种特征图像
 - 对每一个局部滤波结果作一个非线性激励
 - 对每个滤波结果图像作pooling操作，图像大小得到降低，即通过pooling操作，完成了图像下采样 (downsample)
- 对200×200图像、4个5×5卷积核、2×2 pooling：
 - 第一步：得到4个196×196大小的滤波结果
 - 第二步：得到4个196×196大小的非线性激励结果
 - 第三步：得到4个98×98大小的pooling结果

- 第一隐层至第二隐层：假定第二个隐层包含7个图像，采用 3×3 大小的卷积核



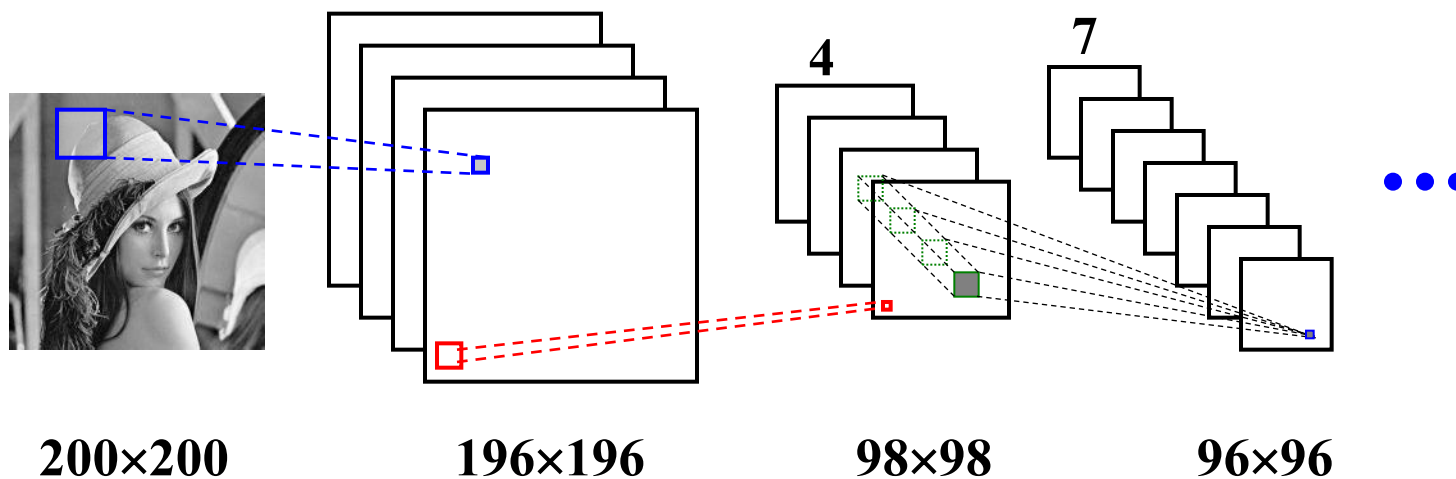
可以将4个输入图像看成一个立方体，使用三维卷积核对立体数据进行卷积操作。此时只需要学习7个三维卷积核，每个包含 $4 \times 3 \times 3 = 36$ 个权重参数。

$$y_j^L = f \left(\sum_i \left(y_i^{L-1} * K_{ij} \right) + w_j \right)$$



6.9.3 卷积神经网络

- 第一隐含层至第二隐含层权重数：
 - 若采用全连接：输入 $\times$ 输出 = $(4 \times 98 \times 98) \times (7 \times 96 \times 96)$
 - 若采用局部连接+权值共享（ 3×3 卷积核）： $9 \times 4 \times 7$



- 网络结构描述

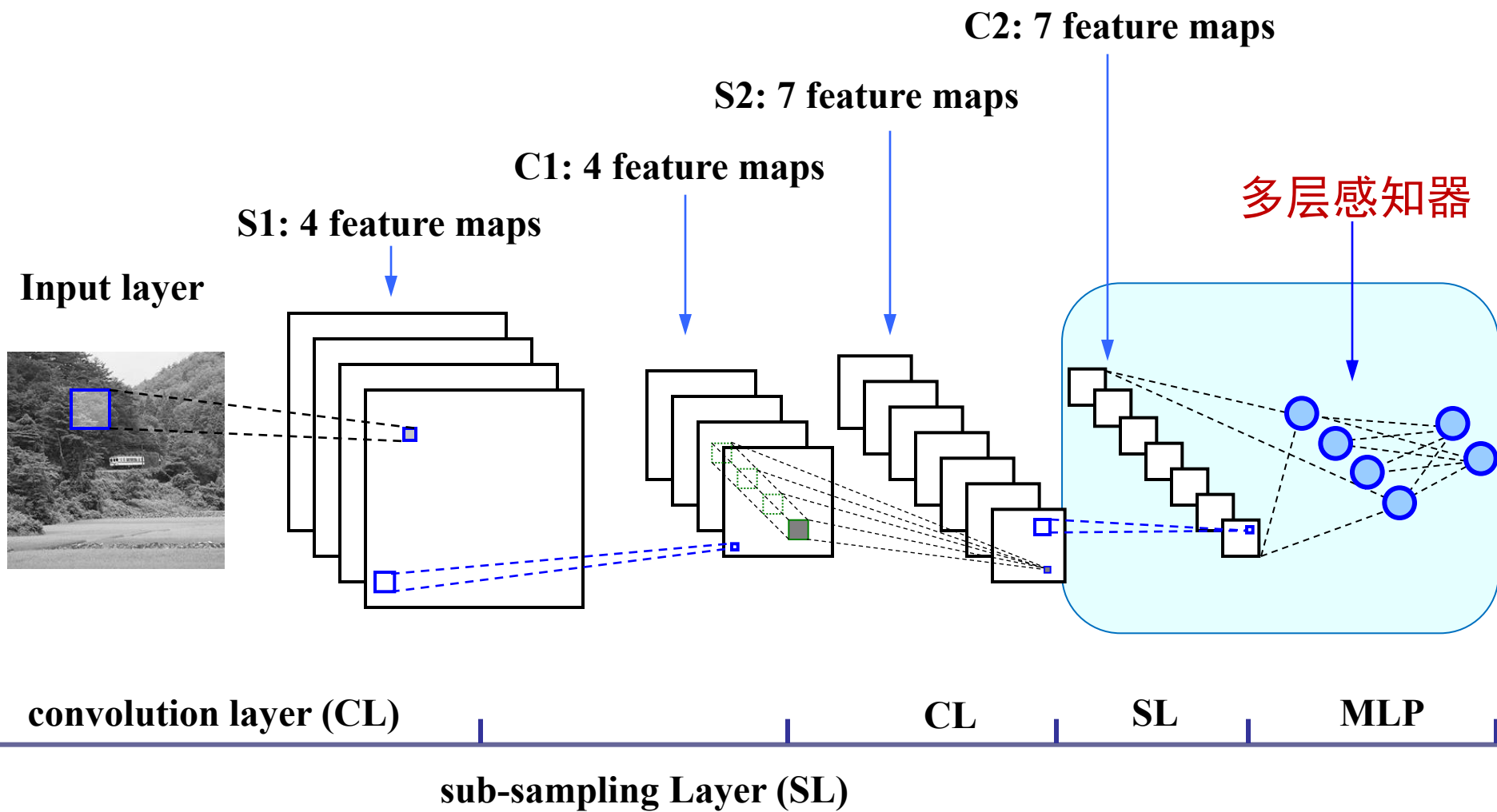
- 图像大小：200×200
- 第一隐含层卷积核大小：5×5
- 第一隐含层图像个数：4
- 第一隐含层图像大小：196 ×196
- 第一隐含层pooling窗口大小：2×2
- 第一隐含层pooling之后图像大小：98×98

- 第二层卷积核大小：3×3
- 第二隐含层图像个数：7
- 第二隐含层图像大小：96 ×96
- 第二隐含层pooling窗口大小：2×2
- 第二隐含层pooling之后图像大小：48×48
- ...

6.9.3 卷积神经网络

- 层与层之间的基本操作
 - 卷积 (Convolution)
 - 将卷积之和加到下一层
 - 对卷积之和进行激励（也可以在pooling之后求激励）
 - 聚合/池化 (Pooling)
 - 下采样，两种基本的运算：
 - 2×2 窗口取平均
 - 2×2 窗口取最大值

整体结构



6.9.3 卷积神经网络

- **网络结构：**
 - 多少个隐含层？
 - 每个隐含层多少个卷积核？
 - 多层感知器里面多少层？
 - 均与实际问题相关

6.9.3 卷积神经网络

- 网络训练

- 采用反向传播算法！
- 选择一个样本 x ，信息从输入层经过逐级的变换，传送到输出层；
- 计算网络实际输出 o 与目标输出 t 的差异；
- 按极小化误差反向传播方法调整权矩阵；
- 也可以采取小的样本组，逐步进行训练。

- 思考题：

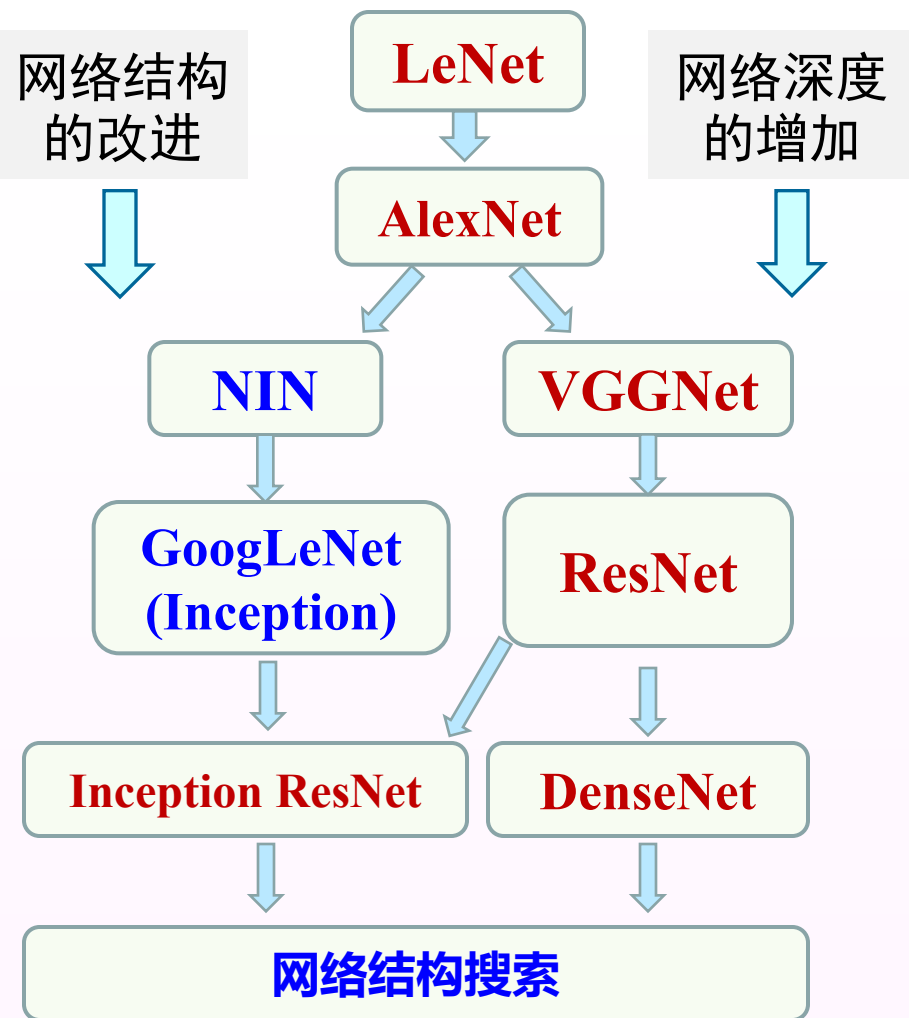
- 权值共享机制下，对权值的梯度如何计算？
- 卷积神经网络中有一个pooling操作，因为这一点需要对现有BP算法做一些修改，如何改？

6.9.3 卷积神经网络

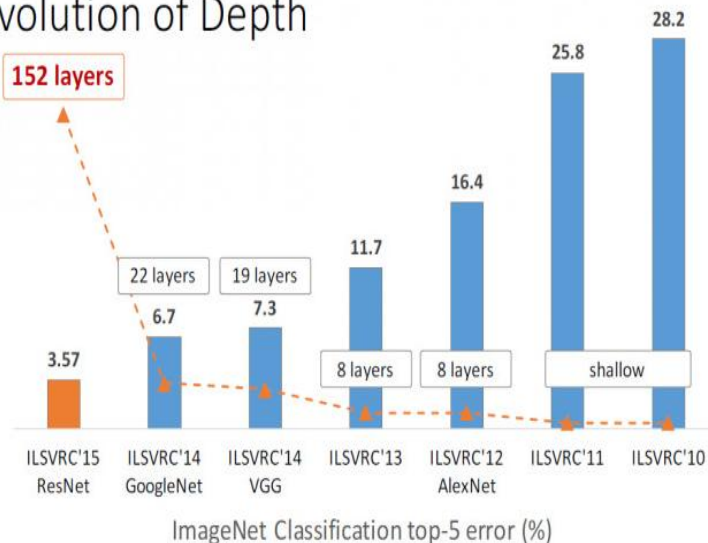
- 核心思想（总结）

- 我们之所以使用卷积后的特征是因为图像具有一种“静态性”的属性，这也就意味着在一个图像区域有用的特征极有可能在另一个区域同样适用。
- 局部感受野、权值共享以及空间下采样这三种结构化思想结合起来获得了某种程度的位移、形变不变性。

深度卷积神经网络发展图

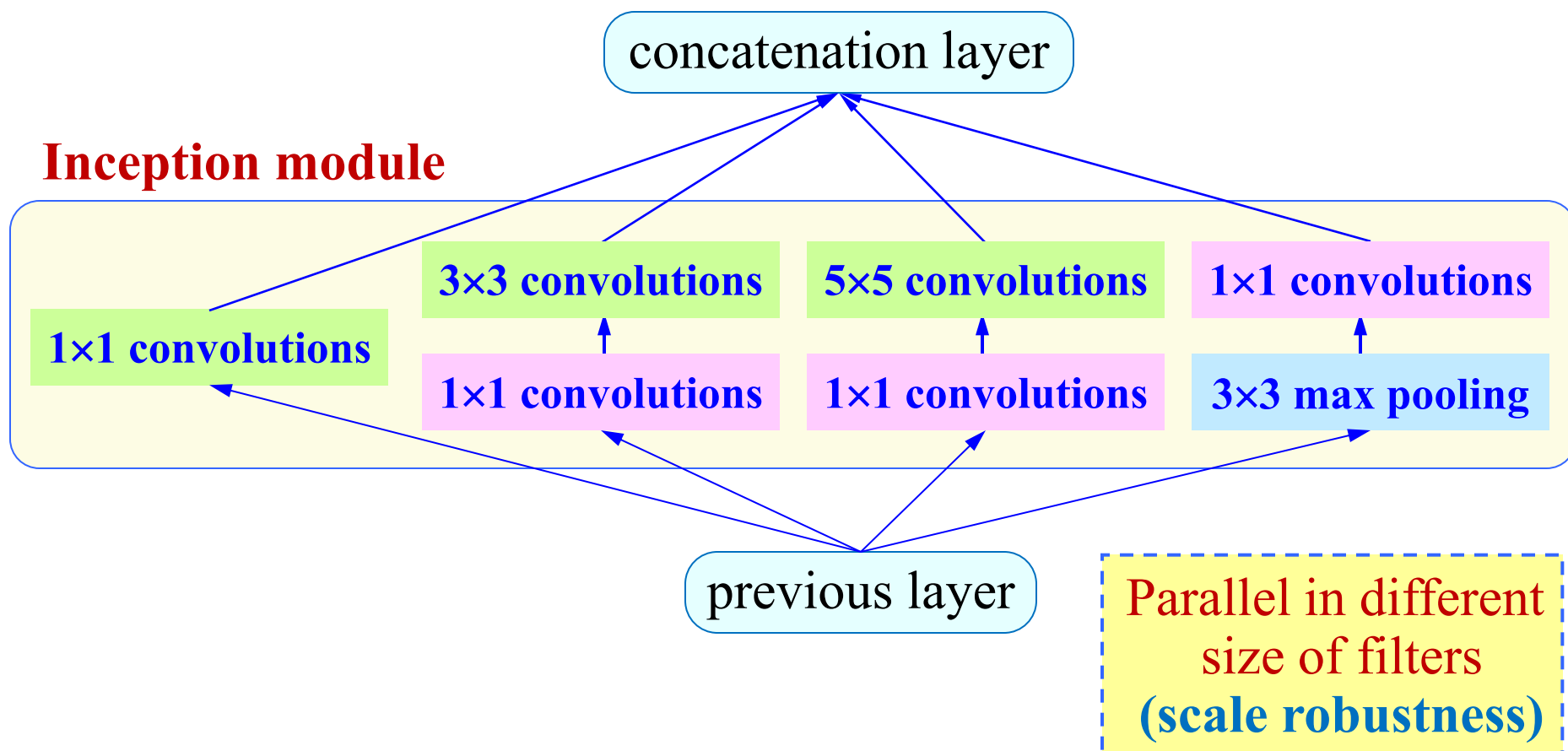


Revolution of Depth



ILSVRC图像分类竞赛结果

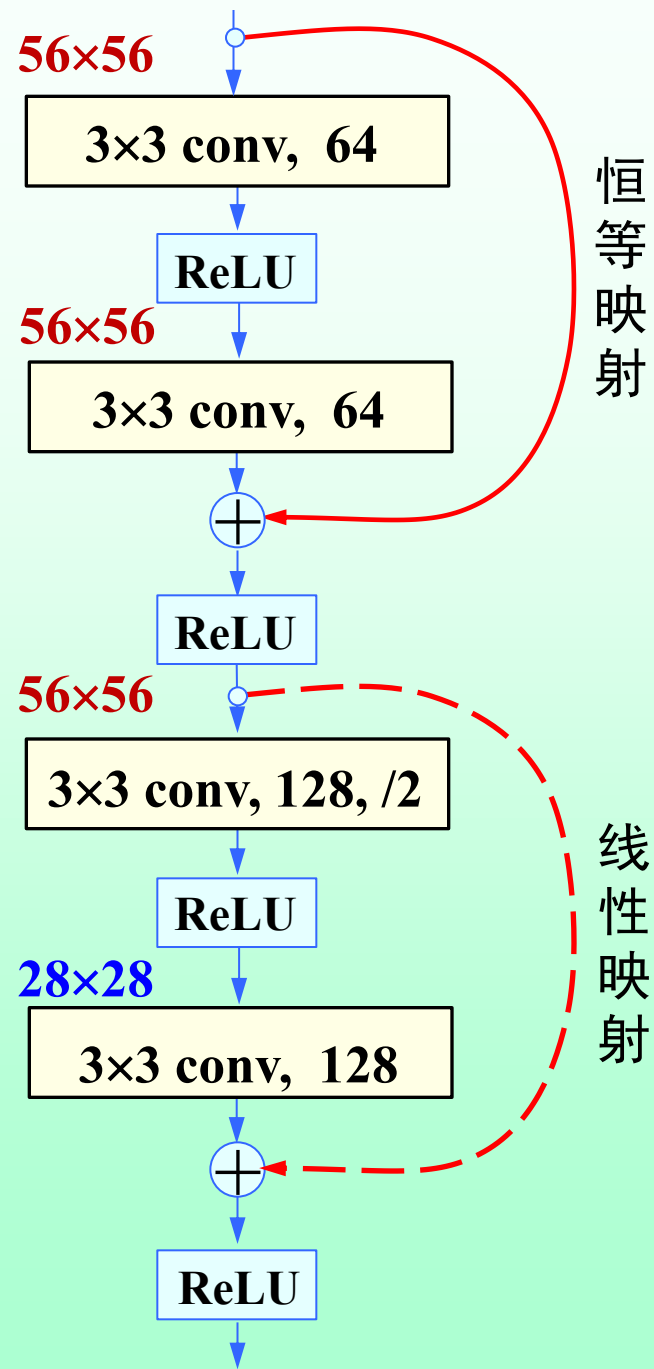
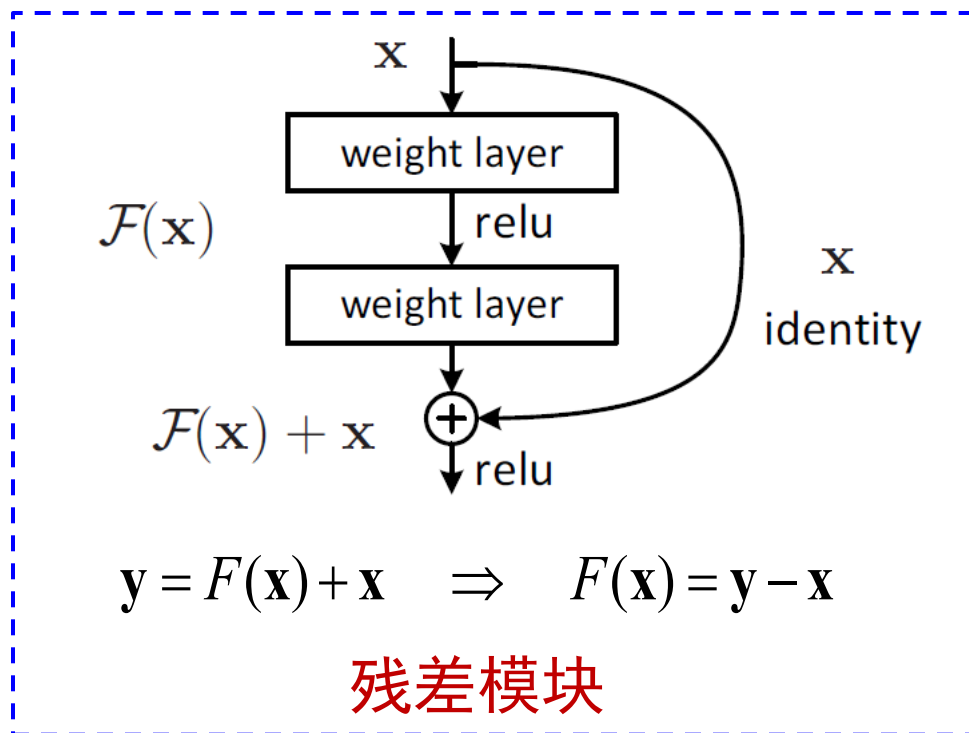
GoogLeNet: 增加网络宽度



- ✓ 多尺度多层次滤波:
- ✓ GoogLeNet的网络参数为AlexNet的1/12, ILSVRC 2014 top-5错误率降至6.67%。

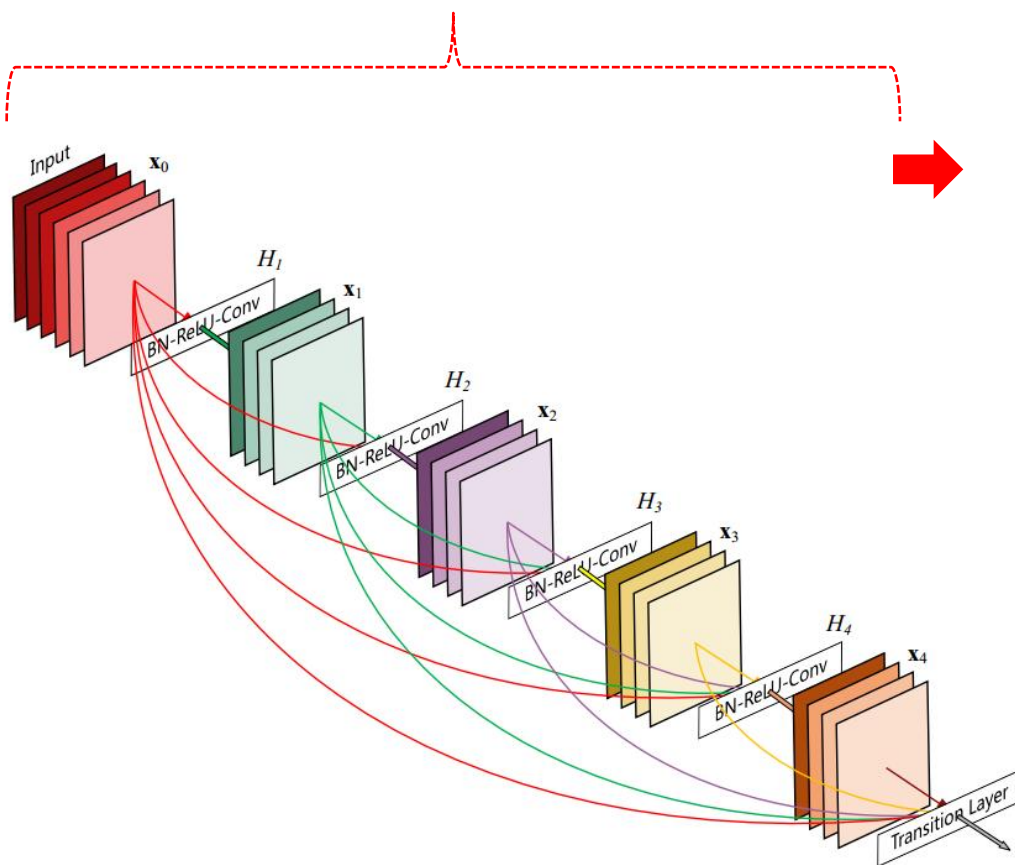
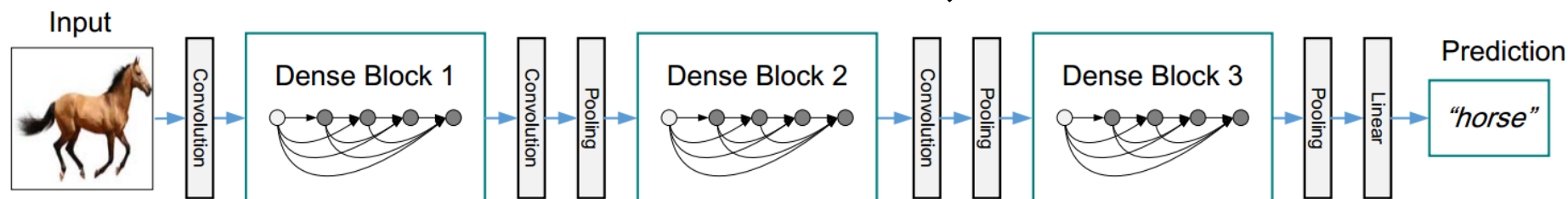
ResNet: 增加了信息传输的高速公路

- 输入通过shortcut与输出相加
- $F(\mathbf{x})$ 为残差映射，线性映射则用于维度匹配。



DenseNet: 密集连接

一个基于DenseNet的分类网络结构示意图，使用了3个dense block



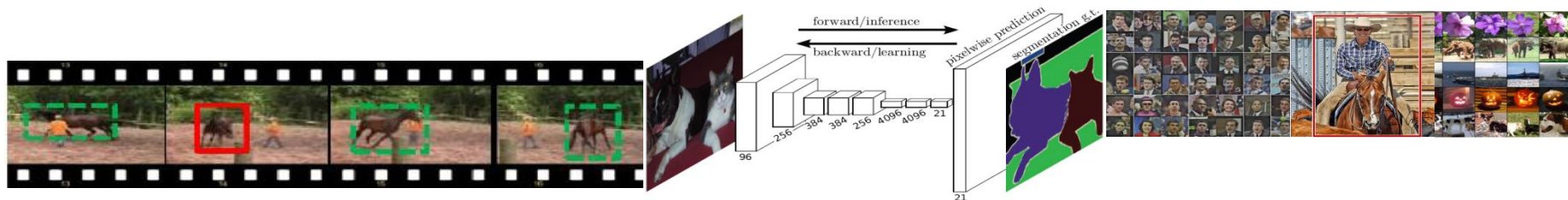
每个dense block有5层，每一层均直接与前面的所有层相连，实现特征的重复利用；同时把网络的每一层设计得特别“窄”，即只学习非常少的特征图(4个)，达到减少参数的作用。

基于DenseNet的分类器只需要ResNet一半的参数量，就可在ImageNet上达到相同分类精度

CNN在计算机视觉中的应用

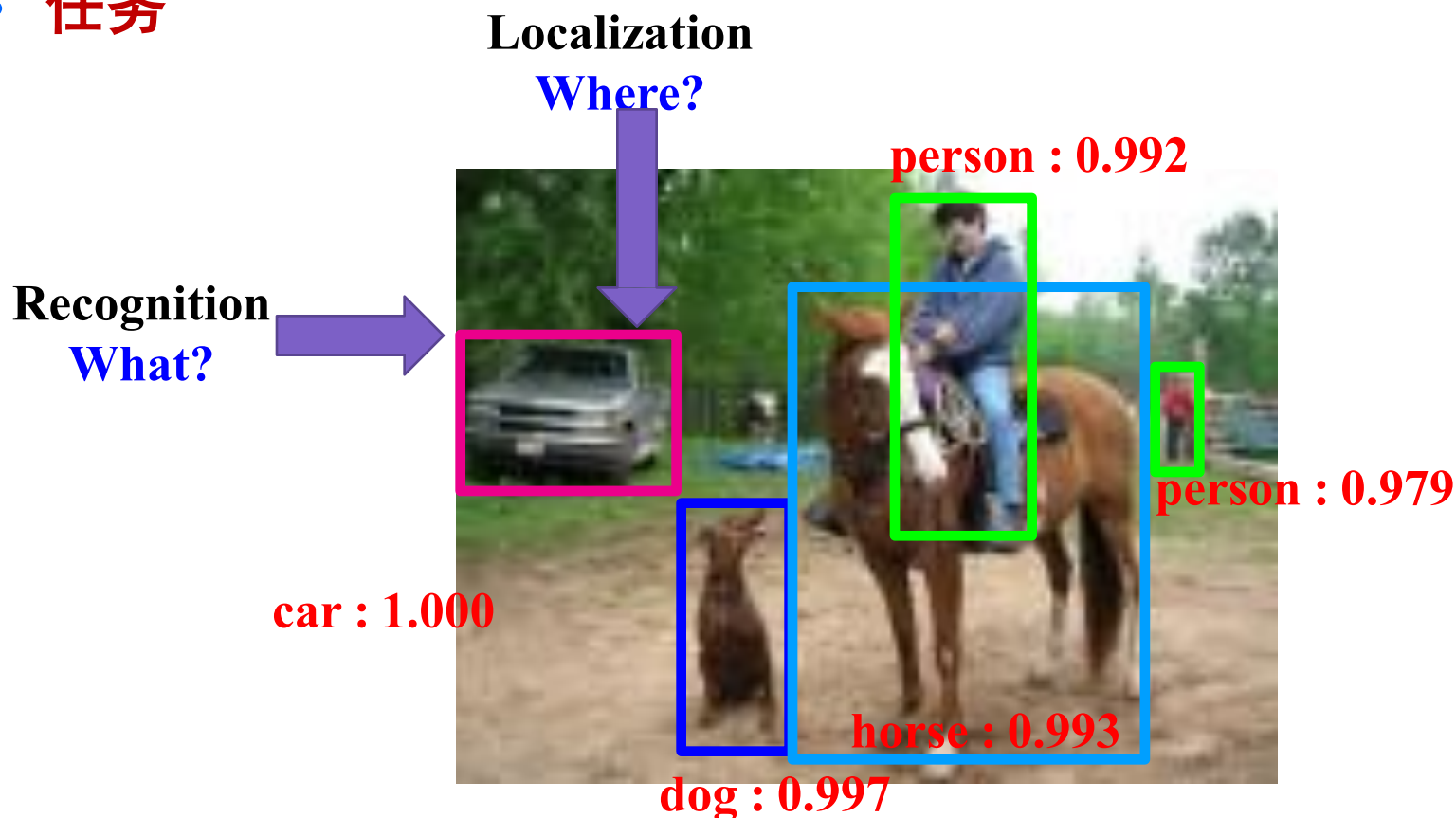
- 随后在计算机视觉领域中的应用全面展开！

- 图像分割、目标检测、图像分类
- 目标跟踪
- 图像标注、图像检索
- 图像描述、问答
- 生物特征识别：人脸识别、人脸检测、指纹识别、虹膜识别
- 行人检测、行人再识别、动作分类、事件分类
- 深度估计、立体匹配、场景解析
- 图像处理：去噪、复原、超分辨率



CNN: 视觉目标检测

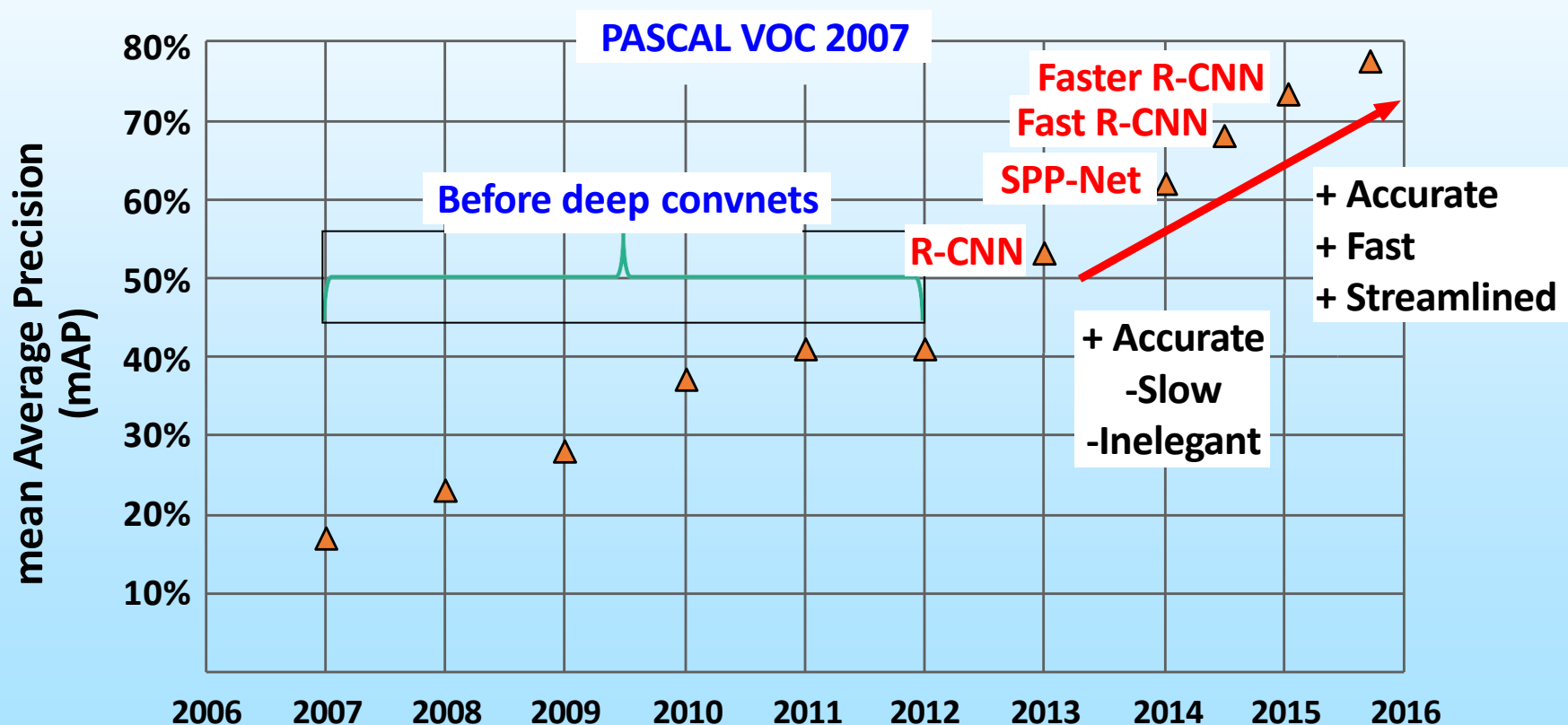
- 任务



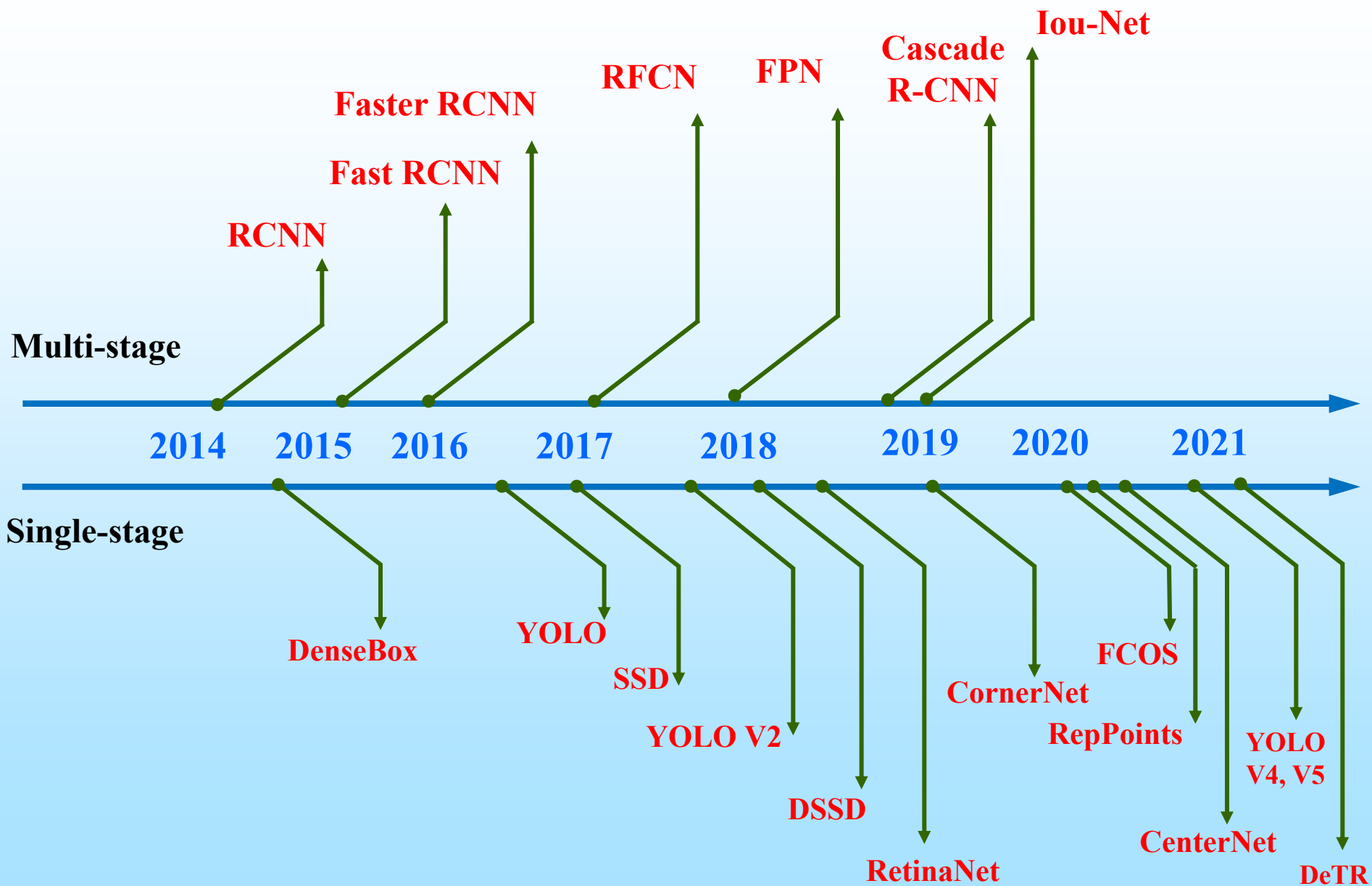
VOC2012: 20 classes. The train data has 11,530 images containing 27,450 ROI annotated objects

CNN: 视觉目标检测

- 目标检测：
 - R-CNN, SPP-Net, Fast R-CNN, Faster R-CNN

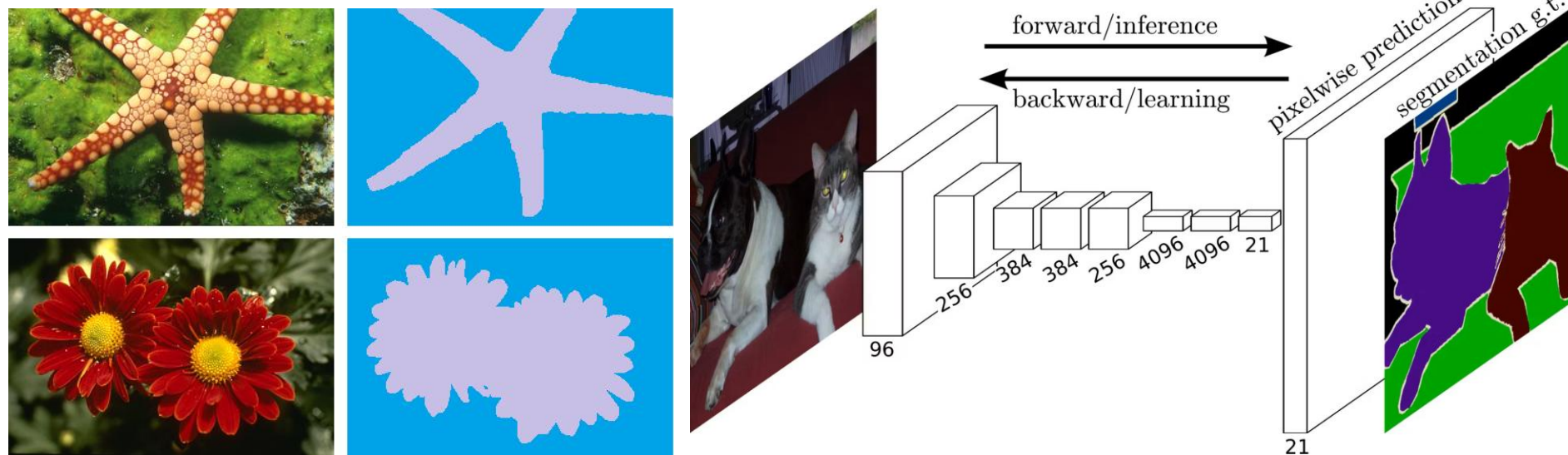


CNN: 视觉目标检测



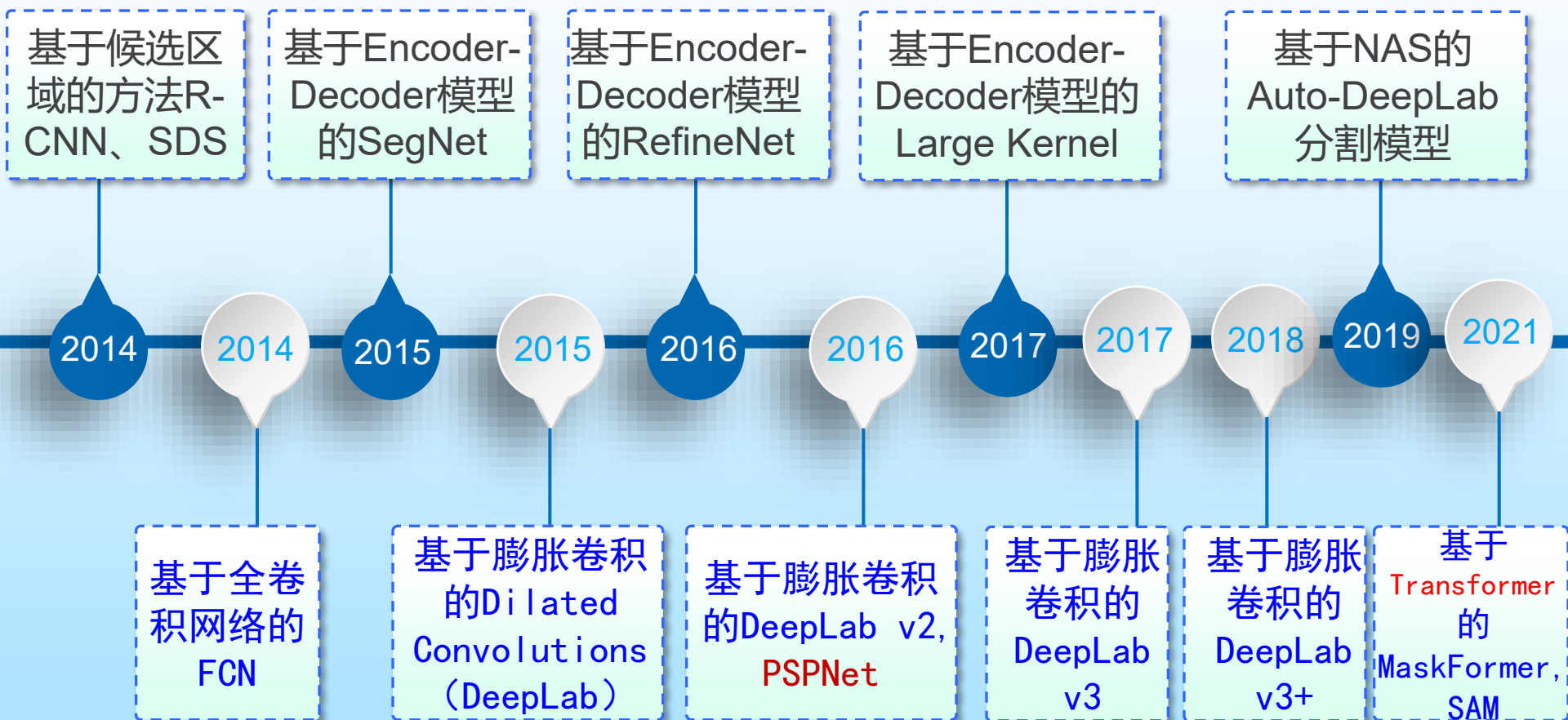
深度学习：图像语义分割

- Image segmentation is a challenging task in computer vision.



Segment image into different regions such that each region has the same semantic meaning.

图像分割：深层模型发展之路



深度学习：图像描述/问答

- Image caption is a very-very challenging task in computer vision.



- ✓ A group of people of Asian descent watch a street performer in a wooded park area.
- ✓ A large crowd of people surround a colorfully dressed street entertainer.
- ✓ A crowd of people watching a balloon twister on a beautiful day.
- ✓ A crowd of people are gathered outside watching a performer.
- ✓ A crowd is gathered around a man watching a performance.

- ✓ O. Vinyals, A. Toshev, S. Bengio, and D. Erhan. Show and tell: A neural image caption generator. CVPR, 2015.
- ✓ K. Xu, J. L. Ba, R. Kiros, K. Cho, A. Courville, R. Salakhutdinov, R. Zemel, Y. Bengio. Show, Attend and Tell: Neural Image Caption Generation with Visual Attention, arXiv: 1502.03044v2, 2015.

Image Caption: 技术之路



6.9.4 自编码器(Autoencoder)

- 背景

- MLP, CNN的训练样本是有标签的。在很多实际应用中，训练样本是没有标签的。
- 自动编码器是一种以无监督方式学习的神经网络，其训练目标是让输出与输入相等，通常用于特征提取。
- 实现这一宏观思想的一种著名神经网络：
 - 2006年，G. E. Hinton和R. R. Salakhutdinov提出的Deep Belief Network (DBN)。
 - 这一思想开创了深度学习浪潮（神经网络的第三次高潮）。

6.9.4 Autoencoder

- **核心思想**

- 自动编码器是一种重构输入信号的神经网络

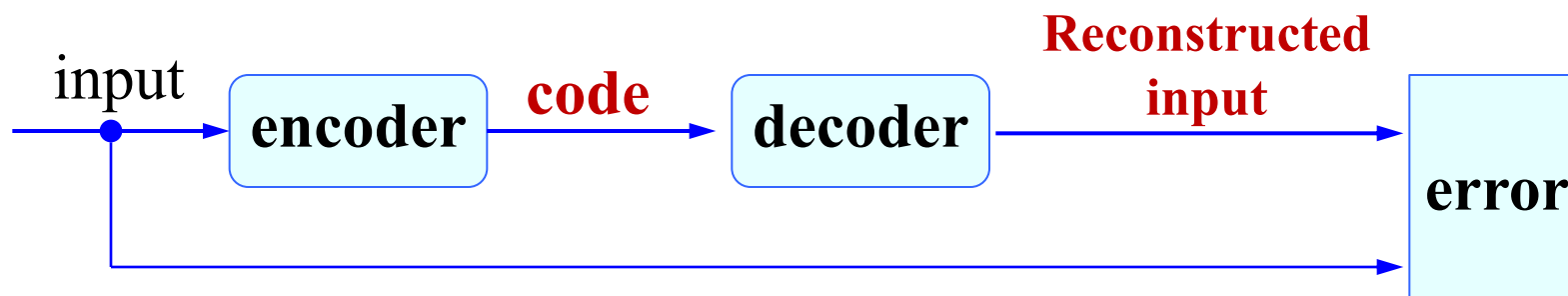
- 由一个编码器(encoder)和一个解码器(decoder)组成
 - 训练过程中, 为了重构输入信号, 网络必须提取到输入数据的最重要的内在因素, 从而实现了**对数据的特征学习**。

- 这也是主成分分析(PCA)的基本原理

- 训练完成后, 编码器输出则可以理解为提取到的数据特征, 因此这是一种典型的**表示学习方法**。

- **网络结构**

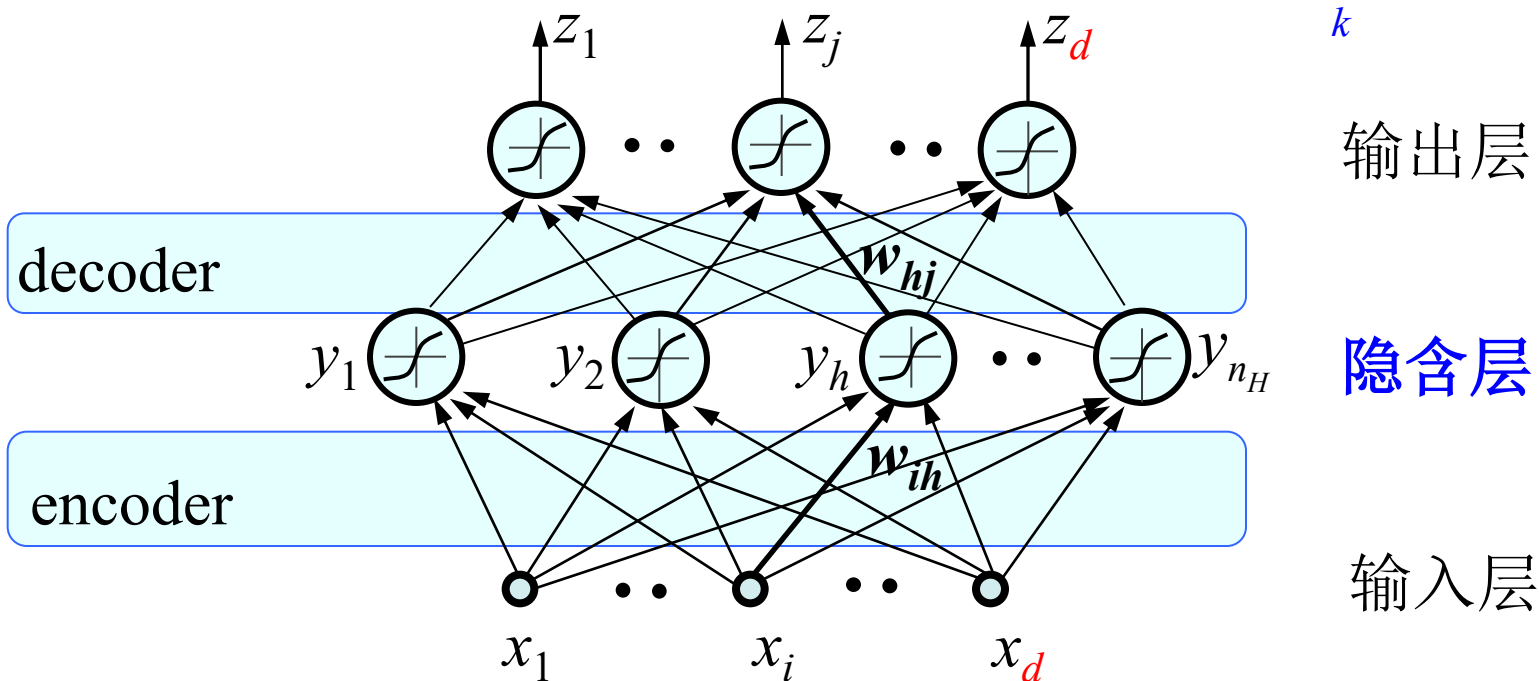
- 将数据输入编码器(encoder), 就会得到一个编码(code)。这个 code 也就是输入的一个表示。
- 将code输入解码器(decoder), 得到对输入数据的重构。
- 理想情况下, 希望 decoder 所输出的信息与输入信号input 是相同的。实际情况下, 会存在误差, 希望这个误差最小。



• 网络表达、第一次编解码训练：

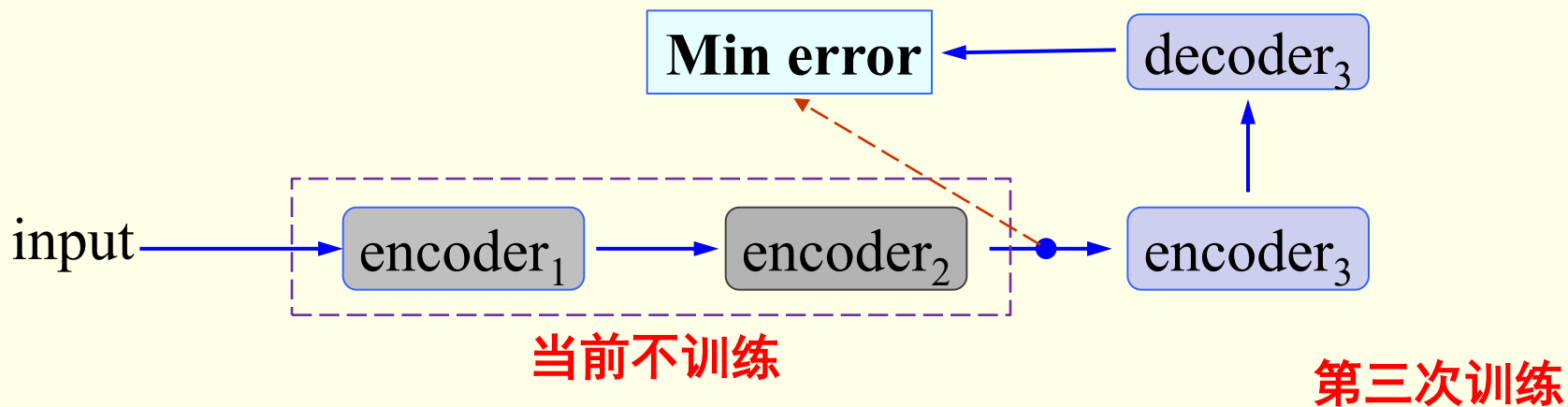
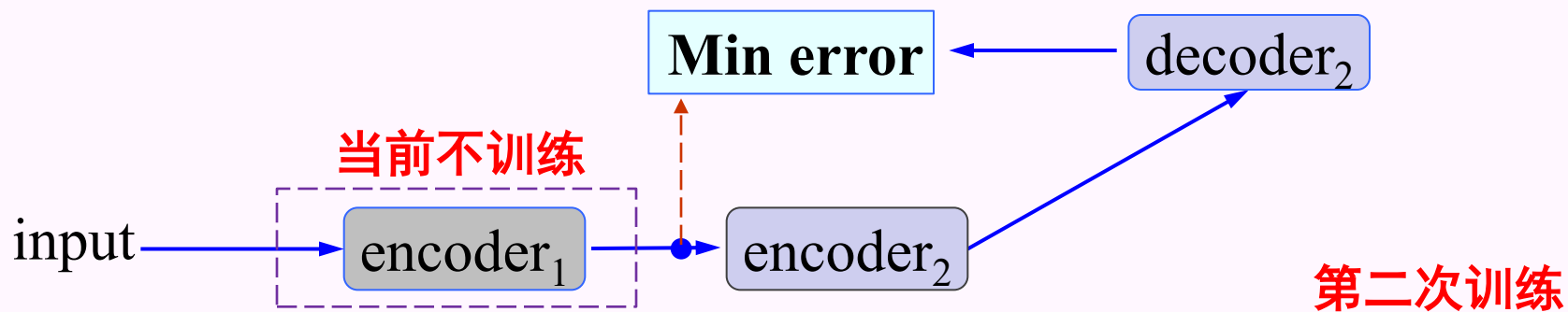
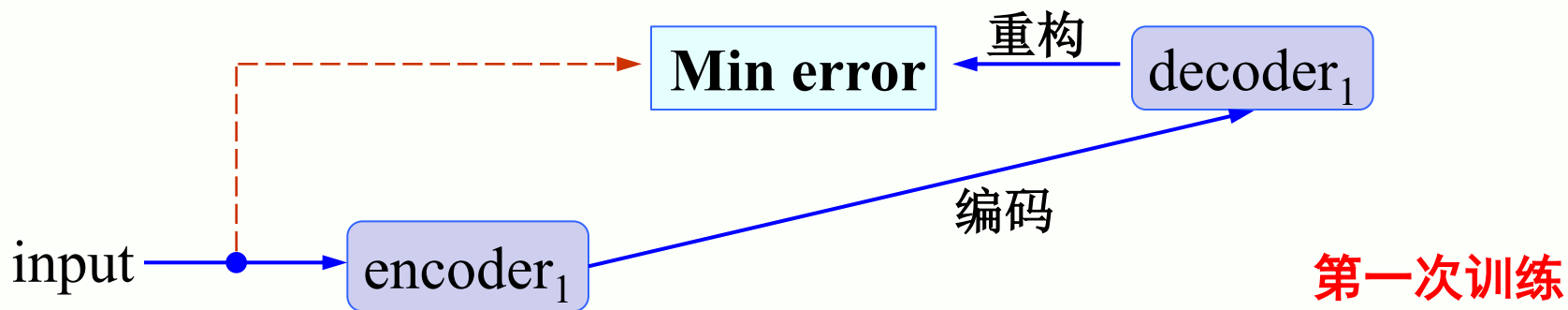
- 编码：建立输入层至隐含层的权重；
- Code：隐含层的输出，也称为表达或特征
- 解码：建立隐含层至输出层的权重

$$\min \sum_k \| \mathbf{x}_k - \mathbf{z}_k \|^2$$



输入层与输出层结点数相等

- 网络训练：（逐层静态进行）

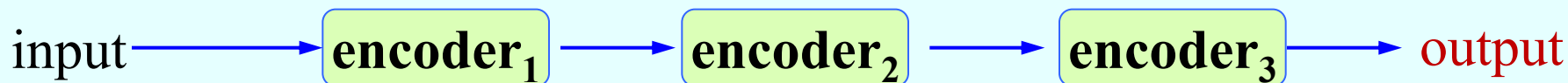


• 网络训练

- 对每个样本，将第一层输出的 code 当成第二层的输入信号，再次利用一个新的三层前向神经网络来最小化重构误差，得到第二层的权重参数（获得第二个编码器）；同时得到样本在该层的code，即原始信号的第二个表达。
- 在训练当前层时，其它层固定不动。完成当前编码和解码任务。前一次“编码”和“解码”均不考虑。
- 因此，这一过程实质上是一个静态的堆叠(stack)过程
- 每次训练可以用BP算法对一个三层前向网络进行训练

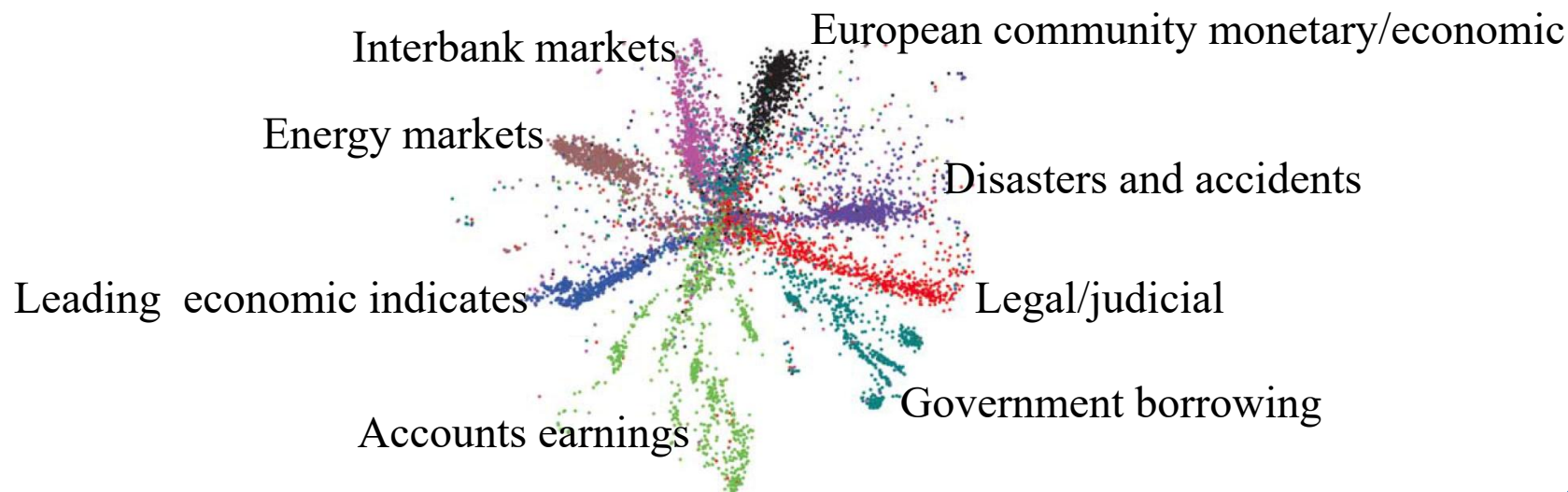
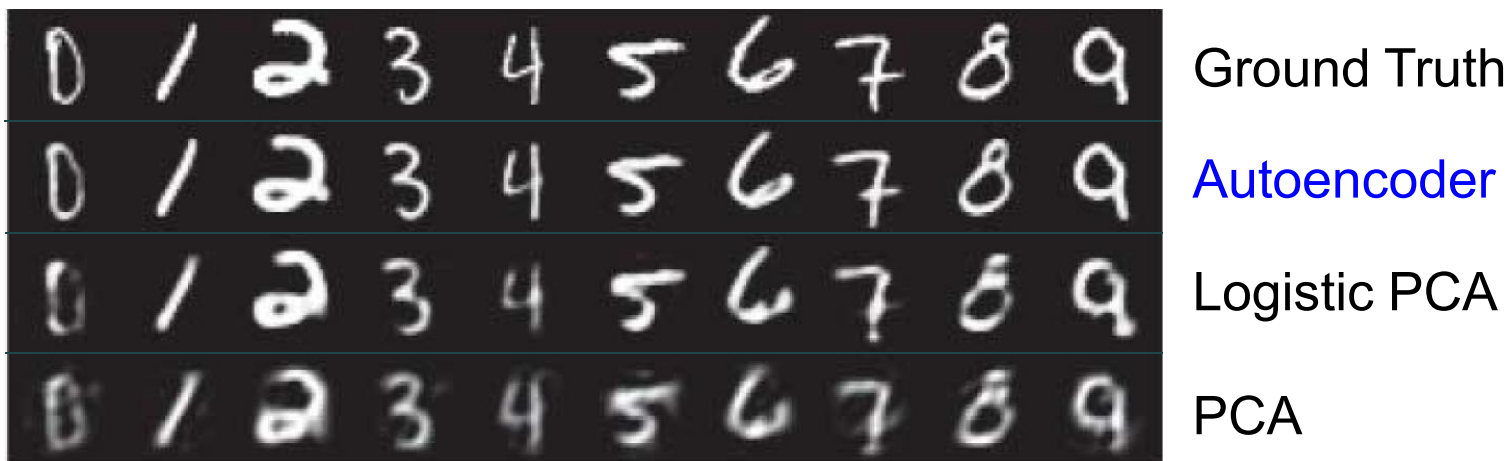
6.9.4 Autoencoder

- 作为特征提取器来使用
 - **只应用编码器**：样本逐步通过多个编码器（层），然后以最后一层编码器的输出作为该样本的新特征。



- 在特征提取应用中，Autoencoder 训练完成后，解码器不予使用。

- 应用举例—重构、降维



6.9.4 Autoencoder

- 扩展 1

- 编码阶段所获得的网络其实质是一种特征学习。层级越高，特征的语义性、抽象性、结构性越明显。
- 对于分类任务，可以事先用 Autoencoder 对数据进行学习。然后以学习得到的权值作为始初权重，采用带有标签的数据对网络进行再次学习，即微调技术(fine-tuning)。
 - 可在编码阶段的最后一层加上一个分类器，比如一个多层感知器（MLP）。

6.9.4 Autoencoder

- 扩展 2

- Autoencoder的输入输出在同一特征空间，训练目标是重构输入数据
- 如果使encoder的输入和decoder的输出对应于不同的数据空间（模态），并引入任务相关的训练目标，则Autoencoder扩展为一般的encoder-decoder框架，该框架在CV/NLP领域极为常用
 - 机器翻译：A语言→B语言
 - 图像描述：图像→语言
 - 语义分割：图像→语义标签
 - 文本图像生成：语言→图像

6.9.4 Autoencoder

- 扩展 3

- Autoencoder的编码器和解码器都是确定性映射
- 如果在decoder的输入层和输出层分别引入随机性，输入层对应隐变量 $\mathbf{h}$ ，输出层对应观测变量 $\mathbf{x}$ ；则decoder扩展为产生式模型 $p(\mathbf{x}|\mathbf{h})$ ，用于描述隐变量已知条件下，观测变量的分布
- encoder的输出对应已知观测变量下隐变量的后验分布 $p(\mathbf{h}|\mathbf{x})$ ，训练结束后，encoder不再使用
- 这就是著名的变分自动编码器VAE

Diederik P Kingma, Max Welling. Auto-Encoding Variational Bayes. ICLR, 2014.

6.9.5 Recurrent Neural Networks

- 前馈神经网络

- 信息向前传递，层内结点之间并不连接，适合于处理静态、独立同分布的数据分析，如回归、分类等任务。

- 反馈神经网络

- 至少包含一个反馈连接的神经网络结构。**网络的输出可以沿着一个回路(loop) 进行流动**。这种网络结构特别适合于处理时序数据。
- 人们通常将反馈神经网络视为一个动态系统，主要关心其随时间变化的动态过程。

6.9.5 Recurrent NN

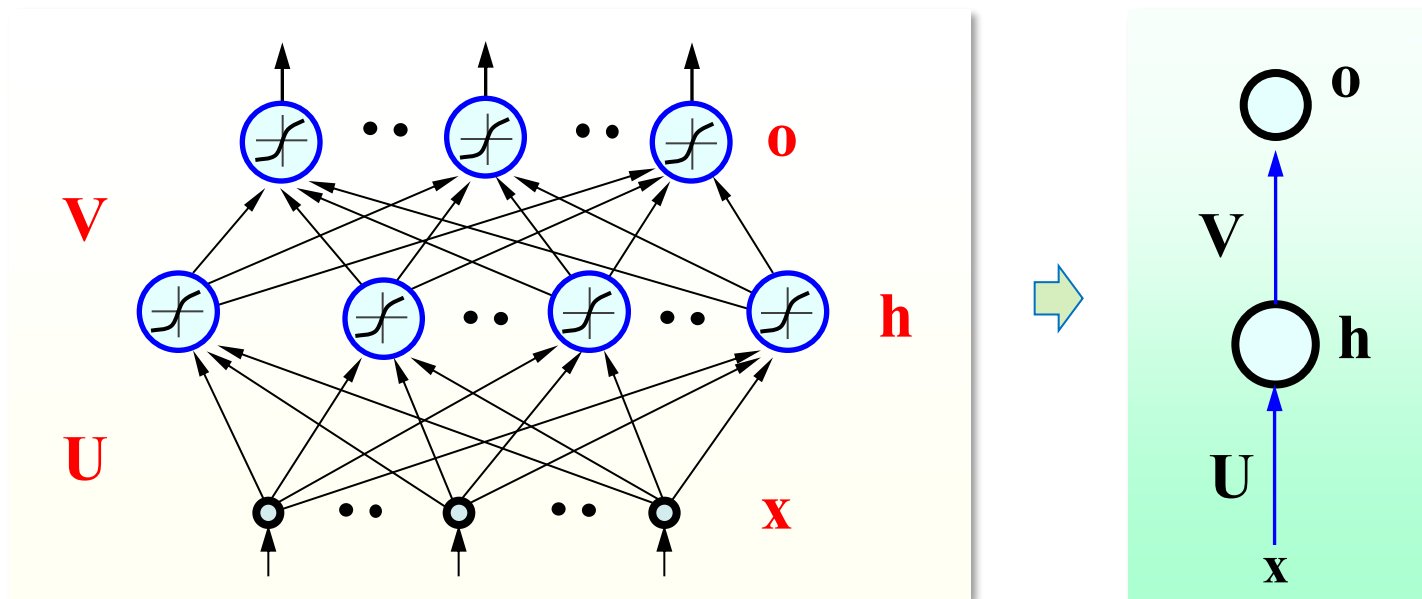
- 几种经典的RNN:

- Hopfield 网络(1982): 主要用于联想记忆而非序列处理, 但它引入了网络内部的循环连接概念, 是 RNN 的先驱。
- Elman 网络(1990): 包含输入层、隐含层和输出层, 将上一时刻的隐含层状态反馈给当前时刻, 这种连接通常称为 recurrent 连接, 是现代 RNN 架构的直接雏形。
 - recurrent连接使得Elman 网络具有检测和产生时变模式的能力
- LSTM网络(1997): 引入了“门控机制” (Gating Mechanism) 和“细胞状态” (Cell State), 巧妙地解决了梯度消失问题, 使得 RNN 能够学习较长的依赖关系。
- Mamba (State Space Models, 2023):

6.9.5 Recurrent NN

- 前馈神经网络

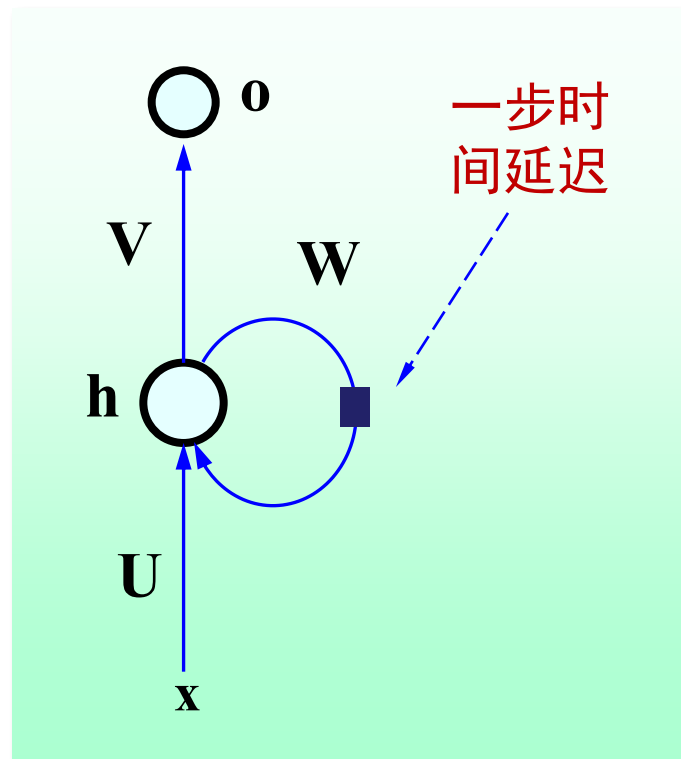
- $\mathbf{x}$: 输入信号 (向量) ; $\mathbf{o}$: 网络输出 (向量)
- $\mathbf{h}$: 隐含层的输出 (向量)
- $\mathbf{U}$: 输入至隐含层的连接权重矩阵 (input to hidden)
- $\mathbf{V}$: 隐含层至输出层的连接权重矩阵 (hidden to output)



6.9.5 Recurrent NN

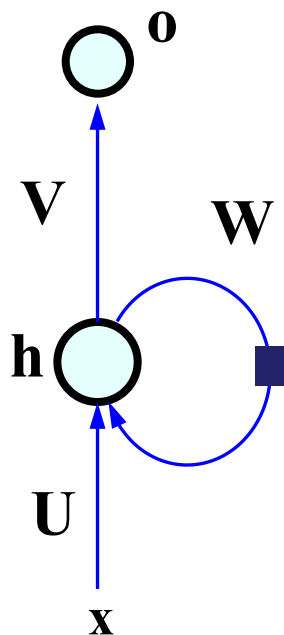
- RNN的基本结构

- x : 输入信号 (向量)
- o : 网络输出 (向量)
- h : 隐含层的输出 (向量)
- U : 输入至隐含层的连接权重矩阵
- V : 隐含层至输出层的连接权重矩阵
- W : 隐含层神经元之间的连接权重矩阵



6.9.5 Recurrent NN

- 网络结构



$$\mathbf{net}_t = \mathbf{b} + \mathbf{W}\mathbf{h}_{t-1} + \mathbf{U}\mathbf{x}_t$$

$$\mathbf{h}_t = f(\mathbf{net}_t)$$

$$\mathbf{o}_t = \mathbf{c} + \mathbf{V}\mathbf{h}_t$$

$\mathbf{b}, \mathbf{c}$: 偏移向量
 $\mathbf{U}, \mathbf{V}, \mathbf{W}$: 权重矩阵
 $f()$: 激励函数, 比如 $\tanh$

网络的功能可以解释为: How the state $\mathbf{h}_t$ at time t captures and summarizes the information from the previous inputs $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_t$.

6.9.5 Recurrent NN

- A simple comparison with Hopfield

RNN:

$$\mathbf{h}_t = f(\mathbf{b} + \mathbf{W}\mathbf{h}_{t-1} + \mathbf{U}\mathbf{x}_t)$$
$$\mathbf{o}_t = \mathbf{c} + \mathbf{V}\mathbf{h}_t$$

Hopfield:

$$\mathbf{h}_0 = \mathbf{x}$$
$$\mathbf{h}_t = f(\mathbf{b} + \mathbf{W}\mathbf{h}_{t-1}), \quad \mathbf{W} = \mathbf{W}^T$$
$$\mathbf{o} = \mathbf{h}_T$$

6.9.5 Recurrent NN

- A simple comparison with MLP

RNN:

$$\mathbf{h}_t = f(\mathbf{b} + \mathbf{W}\mathbf{h}_{t-1} + \mathbf{U}\mathbf{x}_t)$$
$$\mathbf{o}_t = \mathbf{c} + \mathbf{V}\mathbf{h}_t$$

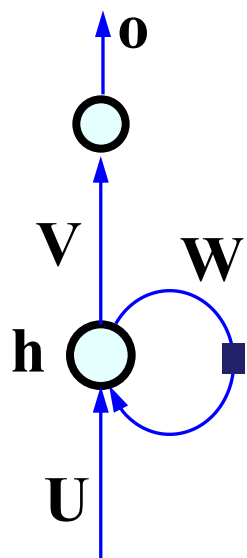
MLP:

$$\mathbf{h} = f(\mathbf{b} + \mathbf{U}\mathbf{x})$$
$$\mathbf{o} = \mathbf{c} + \mathbf{V}\mathbf{h}$$

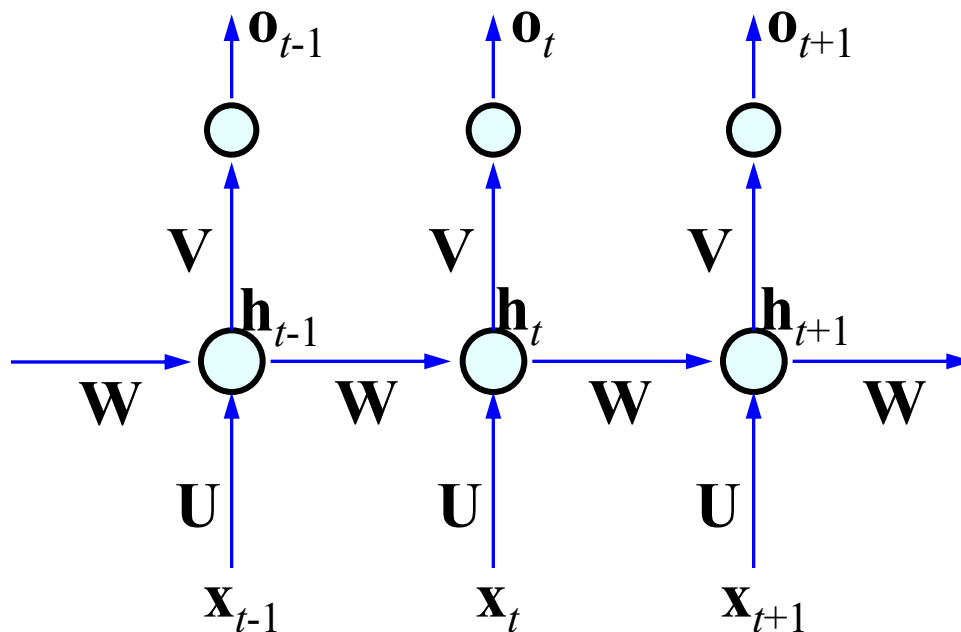
6.9.5 Recurrent NN

- 按时间顺序展开

权值共享: U, V, W

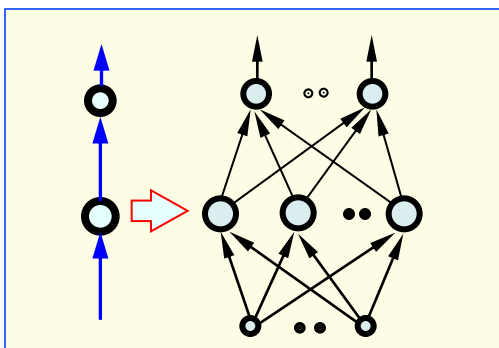


unfold



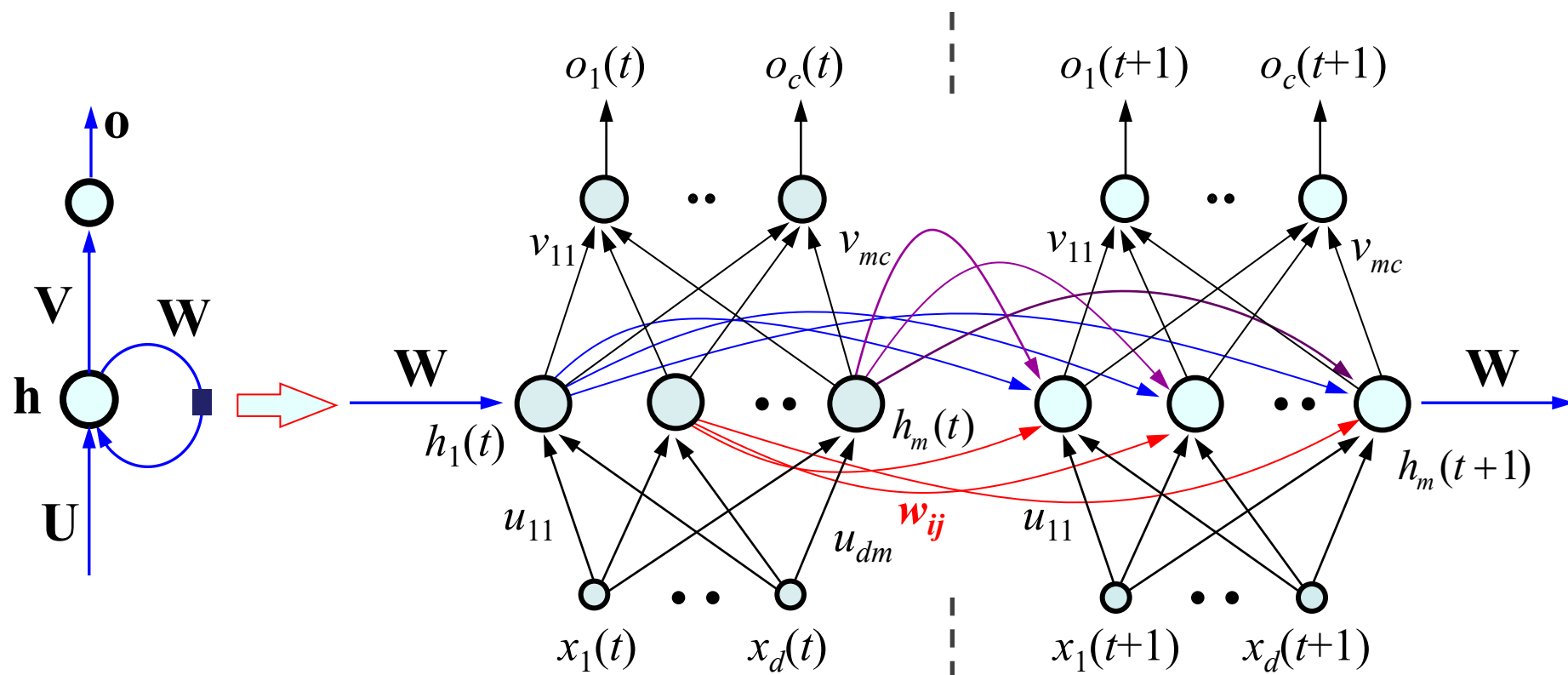
flow graph

$$\mathbf{h}_t = f(\mathbf{b} + \mathbf{W}\mathbf{h}_{t-1} + \mathbf{U}\mathbf{x}_t)$$



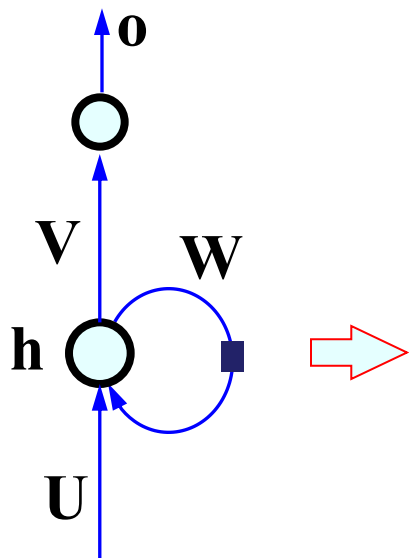
6.9.5 Recurrent NN

- 按时间顺序全结点展开



6.9.5 Recurrent NN

- 网络目标函数（示例：序列标注问题）



$$\text{net}_t = \mathbf{b} + \mathbf{W}\mathbf{h}_{t-1} + \mathbf{U}\mathbf{x}_t$$

$$\mathbf{h}_t = f(\text{net}_t)$$

$$\mathbf{o}_t = \mathbf{c} + \mathbf{V}\mathbf{h}_t$$

$$\hat{\mathbf{y}}_t = \text{soft max}(\mathbf{o}_t)$$

The total loss for a given input/target sequence pair $(\mathbf{x}, \mathbf{y})$ would be the sum of the losses over all the time steps:

$$L(\hat{\mathbf{y}}, \mathbf{y}) = \sum_t l(\hat{y}_t, y_t),$$

$$\text{对分类: } L(\hat{\mathbf{y}}, \mathbf{y}) = -\sum_t \log \hat{y}_{y_t}$$

6.9.5 Recurrent NN

- **Softmax:** 样本分别属于 c 个类别的概率

$$\text{soft max}(\mathbf{o}) = \hat{\mathbf{y}}(\mathbf{o}) = \left(\frac{\exp(o_1)}{\sum_{j=1}^c \exp(o_j)}, \frac{\exp(o_2)}{\sum_{j=1}^c \exp(o_j)}, \dots, \frac{\exp(o_c)}{\sum_{j=1}^c \exp(o_j)} \right)$$

o_j 为输出层第 j 个结点收集到的加权和

- **Softmax loss**

假设数据 $\mathbf{x}$ 对应的类别为 y , y 取 $1, 2, \dots, c$ 中的某个整数。我们的目标就是要最大化 $[\text{softmax}(\mathbf{o})]_y$ 的值 (即第 y 个分量)。

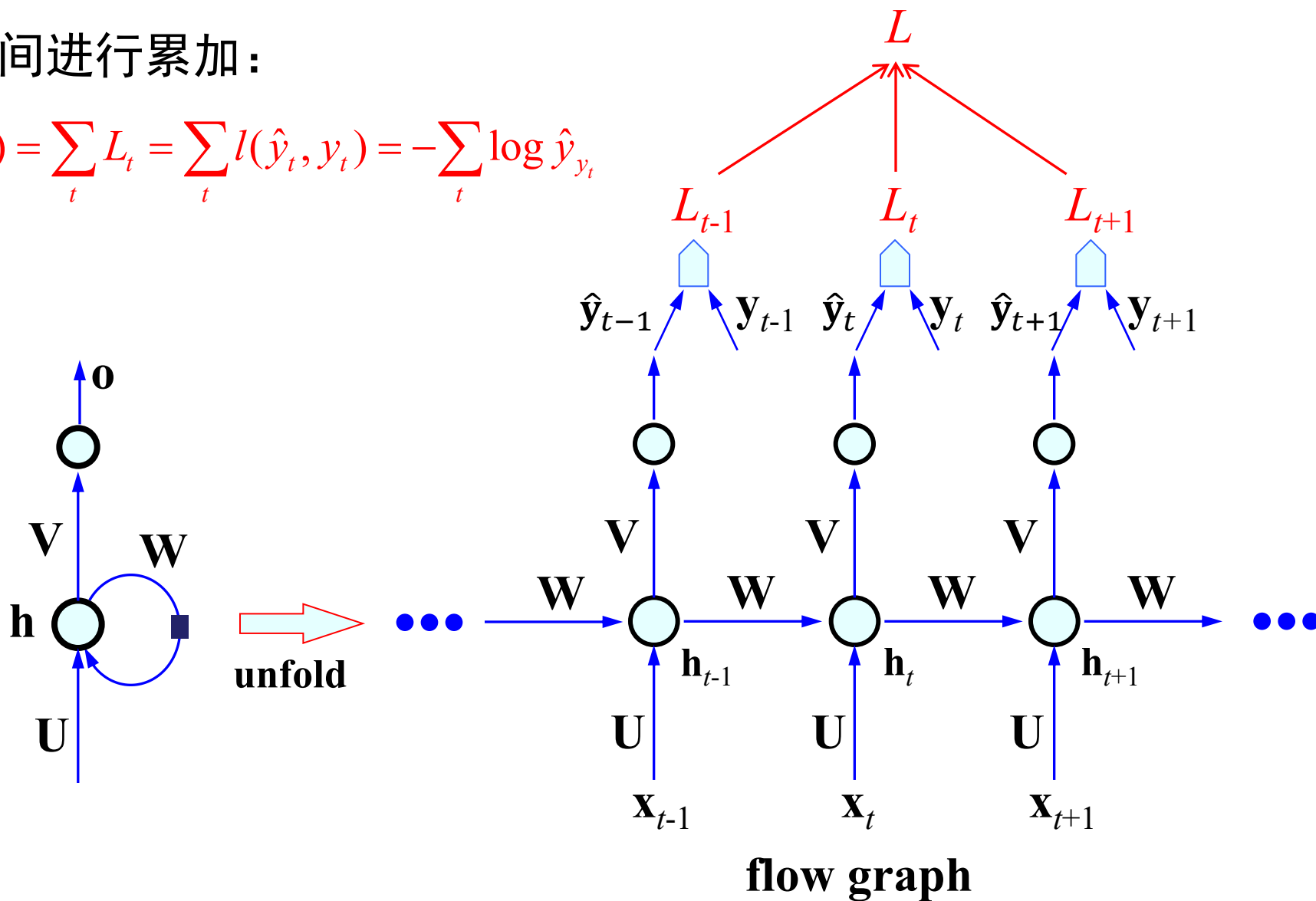
采用负对数似然, 有:

$$l(\hat{\mathbf{y}}, y) = -\log([\hat{\mathbf{y}}(\mathbf{o})]_y) = -\log\left(\frac{e^{o_y}}{\sum_{j=1}^c e^{o_j}}\right) = \log\left(\sum_{j=1}^c e^{o_j}\right) - o_y$$

$[]_y$ 表示向量的第 y 个分量, y 取对应类别的整数

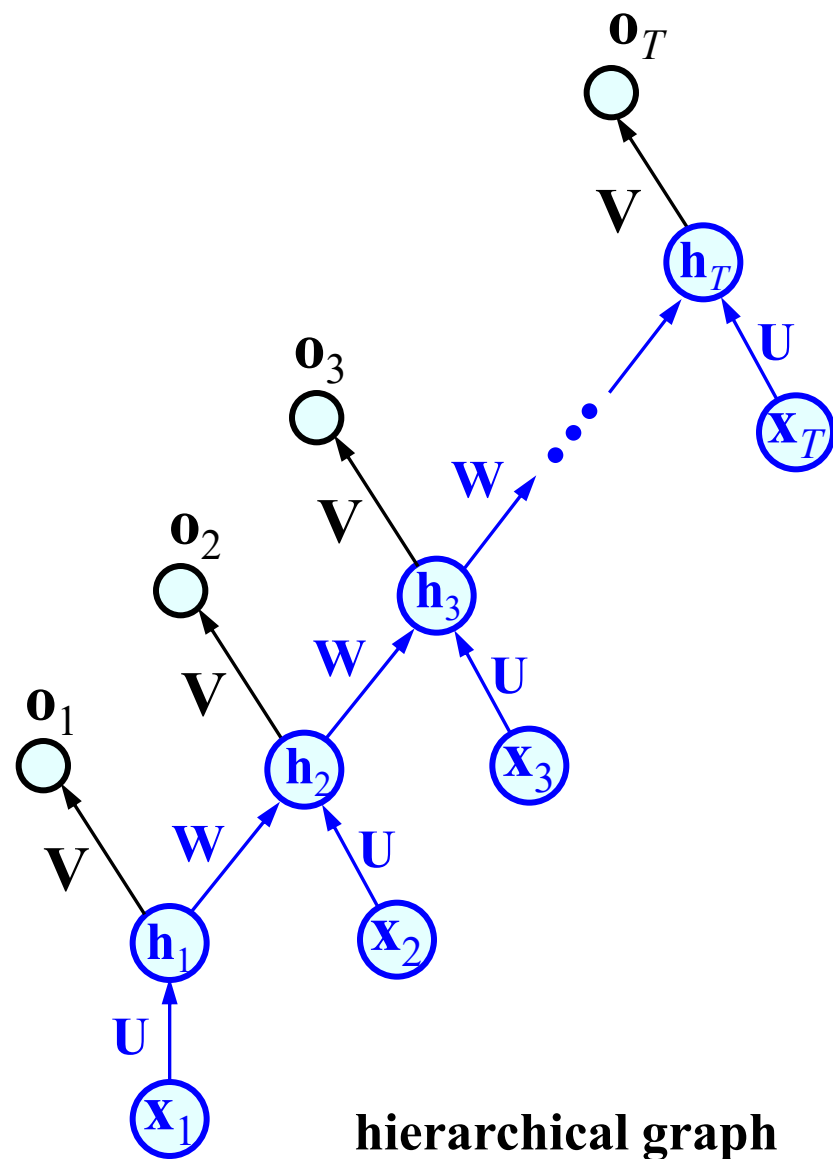
按时间进行累加：

$$L(\hat{\mathbf{y}}, \mathbf{y}) = \sum_t L_t = \sum_t l(\hat{y}_t, y_t) = -\sum_t \log \hat{y}_{y_t}$$



6.9.5 Recurrent NN

- RNN的训练
 - 将RNN展开成多层的前馈网络，然后采用类似的反向传播算法来训练网络。
 - 不同之处在于：层间权重是共享的，是相同的。



- RNN的训练

- Back Propagation Through Time (BPTT)算法:

- 前向计算每个神经元的输出值;
 - 反向计算每个神经元的误差项值, 它是损失函数 L 对各个神经元的加权输入(加权和)的偏导数;
 - 计算每个权重的梯度。
 - 最后, 利用随机梯度下降算法更新权重。

- 以一步时间延迟为例:

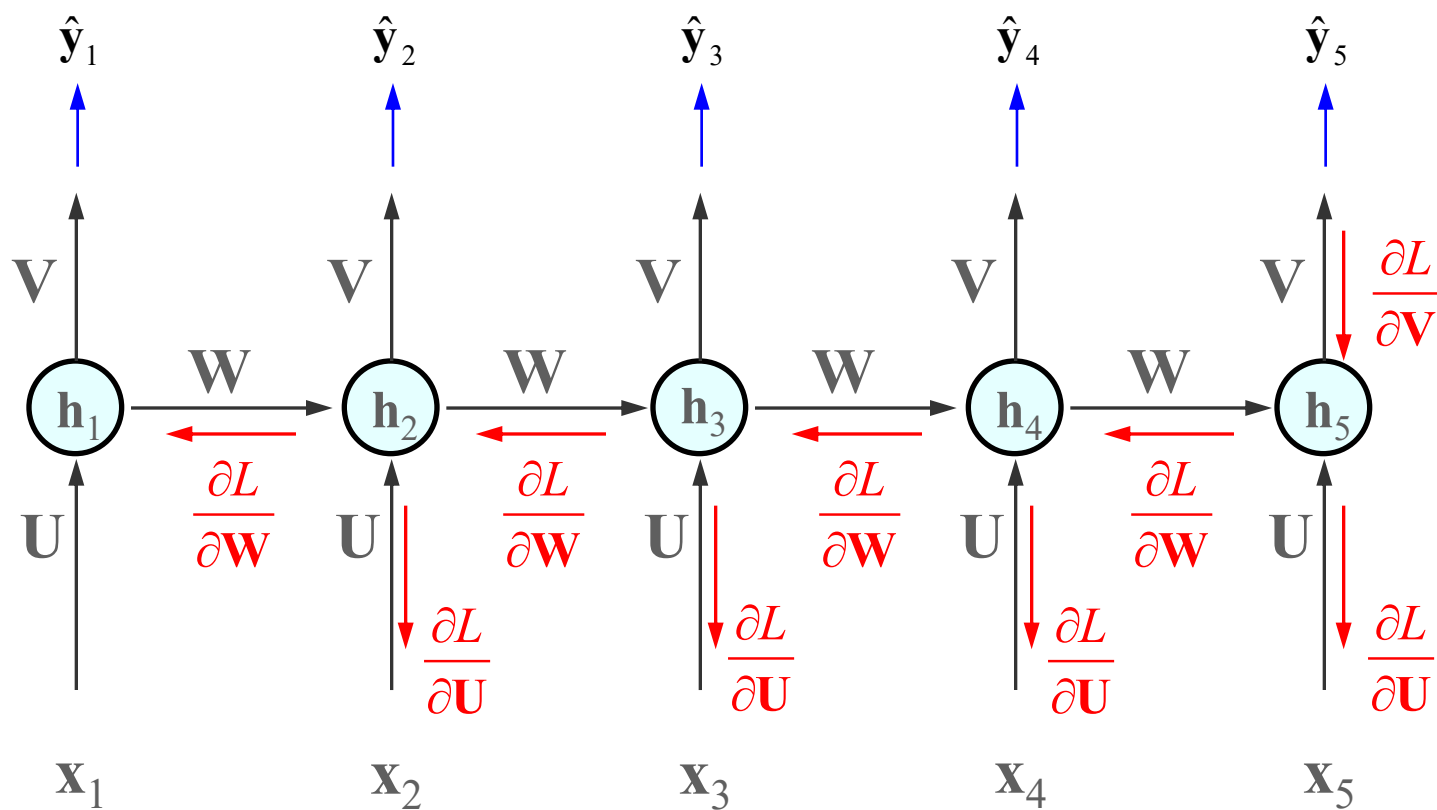
- The nodes in our flow graph will be the sequence of $\mathbf{x}_t$'s, $\mathbf{h}_t$'s, $\mathbf{o}_t$'s, and L_t 's. (输入、隐状态、输出)
 - The parameters are $\mathbf{U}$, $\mathbf{V}$, $\mathbf{W}$, $\mathbf{b}$, $\mathbf{c}$

- 核心任务是计算目标函数对各参数的导数:

$$\nabla_{\mathbf{U}} L, \quad \nabla_{\mathbf{V}} L, \quad \nabla_{\mathbf{W}} L, \quad \nabla_{\mathbf{b}} L, \quad \nabla_{\mathbf{c}} L$$

6.9.5 Recurrent NN

- RNN的训练: BPTT



6.9.6 Long Short-Term Memory (LSTM)

- RNN的问题

传统RNN将激活函数作用于“当前时刻输入信号+上一时刻的隐含结点输出”的仿射变换结果，作为当前隐含结点的输出：

$$\mathbf{h}_t = f(\mathbf{W}\mathbf{h}_{t-1} + \mathbf{U}\mathbf{x}_t + \mathbf{b})$$

- 这种更新方式使 $\mathbf{h}_t$ 随时间 t 发生较大改变，造成历史信息的快速遗忘
- 训练时，梯度容易出现消失或爆炸

recall
$$\frac{\partial E}{\partial \mathbf{net}_l} = f'(\mathbf{net}_l) \odot \mathbf{W}_{l,l+1} \frac{\partial E}{\partial \mathbf{net}_{l+1}}$$

$\odot$: element wise multiplication

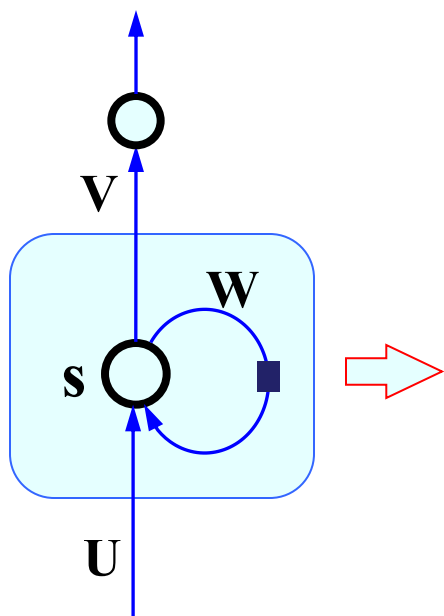
6.9.6 LSTM

- LSTM的核心思想：
 - 通过精细控制对“历史信息（记忆）”的操作，缓解了RNN的遗忘问题和训练问题。
- LSTM的结构
 - LSTM网络将隐含层的单元设计为所谓的LSTM细胞单元
 - 每一个LSTM细胞含有与传统的RNN细胞相同的输入和输出，但它额外包含一个控制信息流动的“门结点系统”。
 - 门系统包含三个部分，除了对LSTM细胞的输入、输出进行加权控制之外，还对记忆（遗忘）进行加权控制

6.9.6 LSTM

- 结构

- 由传统的神经元结构变为带有三个权重门的结构



由简单的输入、输出、隐含层自循环增加了三个门单元：

- 遗忘门：控制对细胞内部状态的遗忘程度
- 输入门：控制对细胞输入的接收程度
- 输出门：控制对细胞输出的认可程度

三个门均由当前输入信号和隐含层前一时刻的输出共同决定

- **输入门 “input gate”**：提供一个**是否接收**当前输入信息的权重(0~1)。
 - 其值取决于当前输入信号和前一时刻隐含层的输出：

$$\mathbf{in}_t = \text{sigmoid}(\mathbf{b}_{in} + \mathbf{U}_{in}\mathbf{x}_t + \mathbf{W}_{in}\mathbf{h}_{t-1})$$

$\mathbf{x}_t$: 当前输入向量;

$\mathbf{h}_{t-1}$: 当前隐含层输出向量, 所有结点在 $t-1$ 时刻的输出;

$\mathbf{b}_{in}$: 输入门的偏移向量;

$\mathbf{U}_{in}$: 输入门的输入权重矩阵;

$\mathbf{W}_{in}$: 输入门的回归权重矩阵;

$\mathbf{in}_t$: $= [in_{t,1}, in_{t,2}, in_{t,3}, \dots]$ (用向量记录所有权重)

- **遗忘门 “forget gate”**：提供一个**是否遗忘**当前记忆的权重(0~1)。
 - 其值取决于当前输入信号和前一时刻隐含层的输出：

$$\mathbf{f}_t = \text{sigmoid}(\mathbf{b}_f + \mathbf{U}_f \mathbf{x}_t + \mathbf{W}_f \mathbf{h}_{t-1})$$

$\mathbf{x}_t$ ：当前输入向量；

$\mathbf{h}_{t-1}$ ：当前隐含层输出向量，包含所有结点的 $t-1$ 时刻输出；

$\mathbf{b}_f$ ：遗忘门的偏移向量；

$\mathbf{U}_f$ ：遗忘门的输入权重矩阵；

$\mathbf{W}_f$ ：遗忘门的回归权重矩阵；

$\mathbf{f}_t$ ： $= [f_{t,1}, f_{t,2}, f_{t,3}, \dots]$ （用向量记录所有权重）

- LSTM 细胞产生的新记忆，由输入与遗忘两部分组成：

产生新记忆：

$$s_t = f_t \otimes \boxed{s_{t-1}} + in_t \otimes \boxed{c_t}$$

读出旧记忆 候选新记忆

$\otimes$: element wise multiplication

$$c_t = \tanh(\mathbf{b} + \mathbf{U}\mathbf{x}_t + \mathbf{W}\mathbf{h}_{t-1})$$

$\mathbf{x}_t$: 当前输入向量；

$\mathbf{h}_{t-1}$: 当前隐含层输出向量，包含所有结点的 $t-1$ 时刻的输出；

$\mathbf{b}$: 正常的输入层至隐层的偏移向量；

$\mathbf{U}$: 正常的输入层至隐层的输入权重矩阵；

$\mathbf{W}$: 正常的输入层至隐层的回归权重矩阵；

贡献候选新记忆

$\mathbf{s}_{t+1}$: $= [s_{t+1,1}, s_{t+1,2}, s_{t+1,3}, \dots]$ (用向量记录所有新记忆)

$\mathbf{c}_t$: $= [c_{t,1}, c_{t,2}, c_{t,3}, \dots]$ (用向量记录所有候选记忆)

- **输出门 “output gate”**：对当前输出提供一个 0~1 之间的权重（对输出的认可程度）。
 - 其值取决于当前输入信号和前一时刻隐含层的输出：

$$\mathbf{o}_t = \text{sigmoid}(\mathbf{b}_o + \mathbf{U}_o \mathbf{x}_t + \mathbf{W}_o \mathbf{h}_{t-1})$$

-
- $\mathbf{x}_t$ ：当前输入向量；
 $\mathbf{h}_{t-1}$ ：当前隐含层输出向量，包含所有结点的 $t-1$ 时刻的输出；
 $\mathbf{b}_o$ ：输出门的偏移向量；
 $\mathbf{U}_o$ ：输出门的输入权重矩阵；
 $\mathbf{W}_o$ ：输出门的回归权重矩阵；
 $\mathbf{o}_t$ ： $= [o_{t,1}, o_{t,2}, o_{t,3}, \dots]$ （用向量记录所有权重）

6.9.6 LSTM

- 隐含层输出

- 隐含层结点的输出由激励后的内部状态（此时称为记忆）与输出门权重共同决定：

$$h_t = o_t \otimes \tanh(s_t)$$

输出门提供的权重

LSTM的当前“记忆”

$\otimes$: element wise multiplication

6.9.6 LSTM

- 结构描述——采用矩阵形式

遗忘门、输入门、输出门：

$$\mathbf{f}_t = \text{sigmoid}(\mathbf{b}_f + \mathbf{U}_f \mathbf{x}_t + \mathbf{W}_f \mathbf{h}_{t-1})$$

$$\mathbf{in}_t = \text{sigmoid}(\mathbf{b}_{in} + \mathbf{U}_{in} \mathbf{x}_t + \mathbf{W}_{in} \mathbf{h}_{t-1})$$

$$\mathbf{o}_t = \text{sigmoid}(\mathbf{b}_o + \mathbf{U}_o \mathbf{x}_t + \mathbf{W}_o \mathbf{h}_{t-1})$$

候选记忆（新贡献部分）：

$$\mathbf{c}_t = \tanh(\mathbf{b} + \mathbf{U} \mathbf{x}_t + \mathbf{W} \mathbf{h}_{t-1})$$

Cell的新记忆：

$$\mathbf{s}_t = \mathbf{f}_t \otimes \mathbf{s}_{t-1} + \mathbf{in}_t \otimes \mathbf{c}_t$$

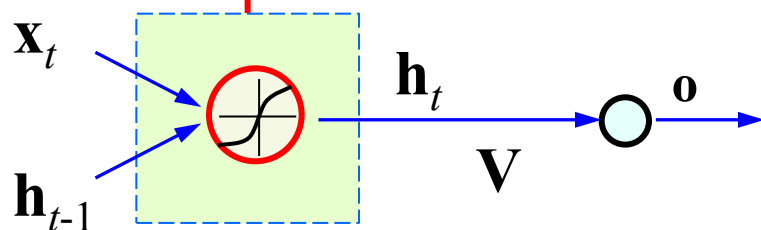
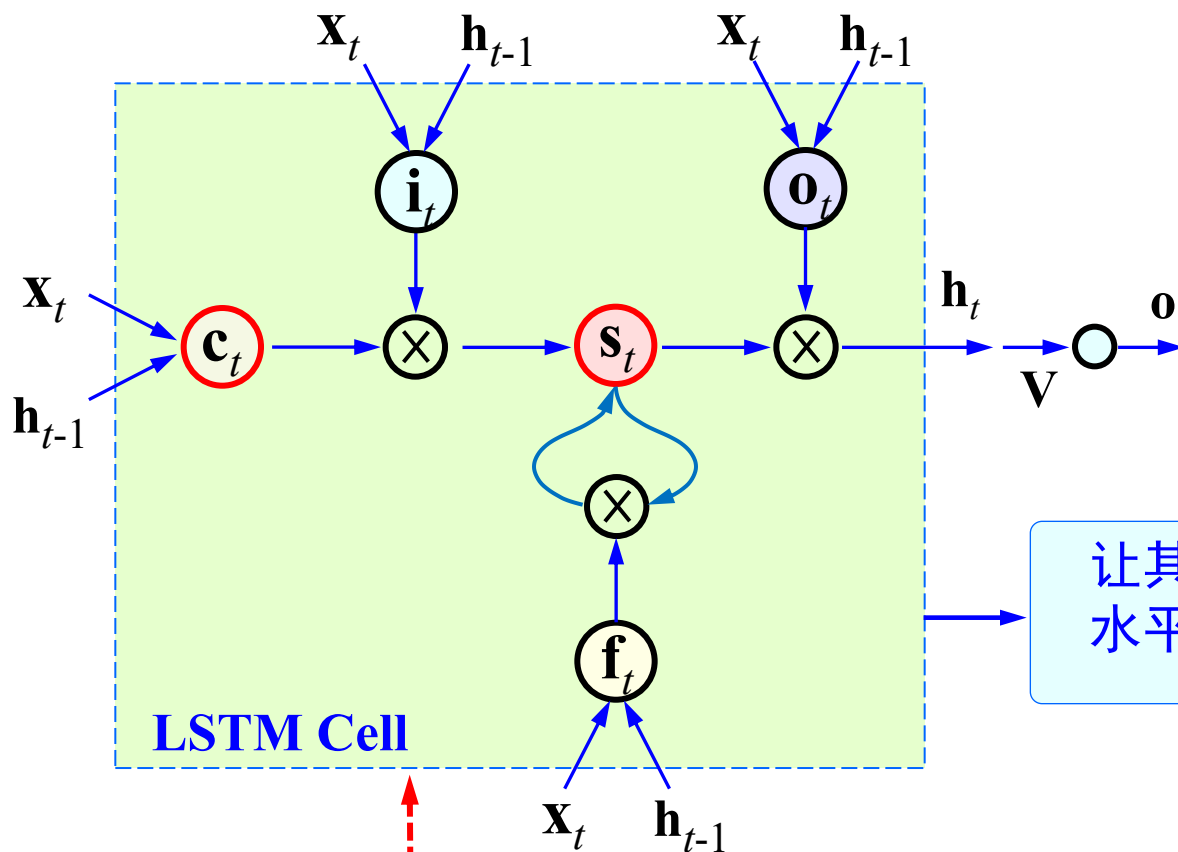
Cell的输出：

$$\mathbf{h}_t = \mathbf{o}_t \otimes \tanh(\mathbf{s}_t)$$

网络的输出：

$$\mathbf{z}_t = \text{softmax}(\mathbf{V} \mathbf{h}_t + \mathbf{c})$$

$\otimes$: element wise multiplication



传统RNN

$$\frac{\partial L_t}{\partial \mathbf{s}_k} \approx \left(f'(\mathbf{s}) \mathbf{W} \right)^t \frac{\partial L_t}{\partial \mathbf{s}_{k+t}}$$

- 网络训练

- 估计如下矩阵或向量

$(\mathbf{U}_{in}, \mathbf{W}_{in}, \mathbf{b}_{in}), (\mathbf{U}_o, \mathbf{W}_o, \mathbf{b}_o), (\mathbf{U}_f, \mathbf{W}_f, \mathbf{b}_f), (\mathbf{U}, \mathbf{W}, \mathbf{b}), (\mathbf{V}, \mathbf{c})$

- 基于两点：

- 基于信息流动的方式，采用反向传播算法
 - 计算目标函数、隐状态(t)对各变量的偏导数

- LSTM并非完全解决了遗忘、梯度消失/扩散的问题，只是有所缓和。仍可考虑如下技术：

- 充分利用二阶梯度的信息（但通常并不鼓励）
 - 更好的初始化
 - 动量机制
 - 对梯度的模或者元素作一些裁剪
 - 正则化—鼓励信息流动

6.9.7 Transformer

- RNN的问题

- 由于RNN、LSTM模型的序列属性，在更新过程中必然逐渐丢失历史信息

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$$

例如： h_{100} 中只残留很少的 x_1 信息

- 但在一些复杂的NLP任务中，长距依赖非常重要且普遍，这成为制约RNN、LSTM性能的关键
- 如何设计有效的机制对长距依赖关系建模？
 - 注意力机制 (attention mechanism)

D. Bahdanau, etc. Neural machine translation by jointly learning to align and translate. ICLR, 2015.
A. Vaswani, etc. Attention is all you need. NIPS, 2017.

6.9.7 Transformer

- 注意力机制的基本原理

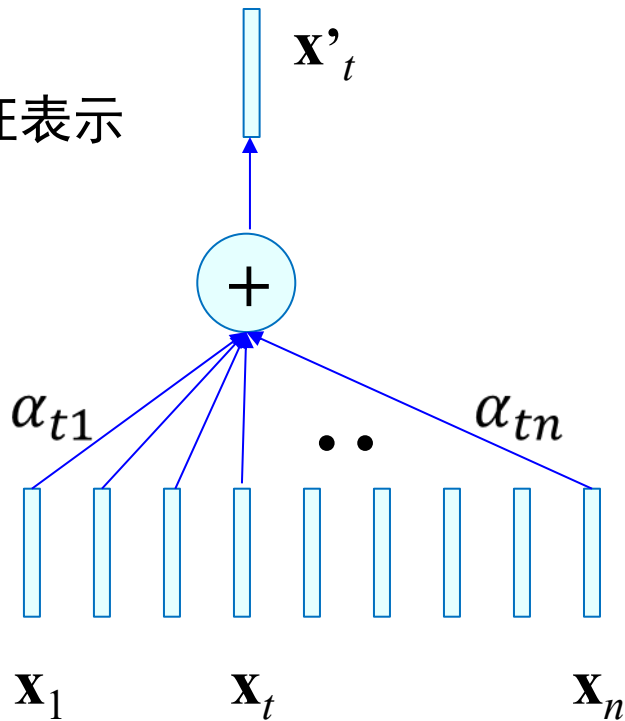
对于输入序列 $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$ ，计算 $\mathbf{x}_t$ 的特征表示

- $\mathbf{x}_t$ 与 $\mathbf{x}_i$ 的相似度

$$s_{ti} = \mathbf{x}_t^T \mathbf{x}_i \quad i = 1, \dots, n$$

- 注意力系数
$$\alpha_{ti} = \frac{\exp(s_{ti})}{\sum_{j=1}^n \exp(s_{tj})}$$

- 加权和
$$\mathbf{x}_t' = \sum_{i=1}^n \alpha_{ti} \mathbf{x}_i$$



- ✓ 注意力系数反应数据相关性的强弱
- ✓ 按注意力系数对所有数据求和（凸组合）
- ✓ 实现了全局感受野，是Transformer的核心

6.9.7 Transformer

- 单头自注意力(single head self-attention)

实际中, 引入三个线性变换 $\mathbf{W}^Q$, $\mathbf{W}^K$, $\mathbf{W}^V$

Query: $\mathbf{W}^Q \mathbf{x}_t$

Keys: $\{ \mathbf{W}^K \mathbf{x}_i \}$

Values: $\{ \mathbf{W}^V \mathbf{x}_i \}$

- $\mathbf{x}_t$ 与 $\mathbf{x}_i$ 的相似度

Scaled Dot-Product Attention

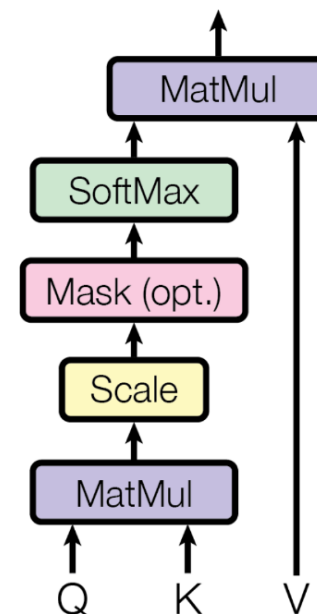
$$s_{ti} = (\mathbf{W}^Q \mathbf{x}_t)^T (\mathbf{W}^K \mathbf{x}_i)$$

- 注意力系数

$$\alpha_{ti} = \frac{\exp(s_{ti})}{\sum_{j=1}^n \exp(s_{tj})}$$

- 加权和

$$\mathbf{x}_t' = \sum_{i=1}^n \alpha_{ti} (\mathbf{W}^V \mathbf{x}_i)$$



6.9.7 Transformer

- 多头自注意力(multi-head self-attention)

为进一步提升模型能力, 引入多组注意力参数 $\{\mathbf{W}_h^Q, \mathbf{W}_h^K, \mathbf{W}_h^V\}$, $h = 1, \dots, H$

- 注意力系数

$$\alpha_{ti}^h = \frac{\exp((\mathbf{W}_h^Q \mathbf{x}_t)^T (\mathbf{W}_h^K \mathbf{x}_i))}{\sum_{j=1}^n \exp((\mathbf{W}_h^Q \mathbf{x}_t)^T (\mathbf{W}_h^K \mathbf{x}_j))}$$

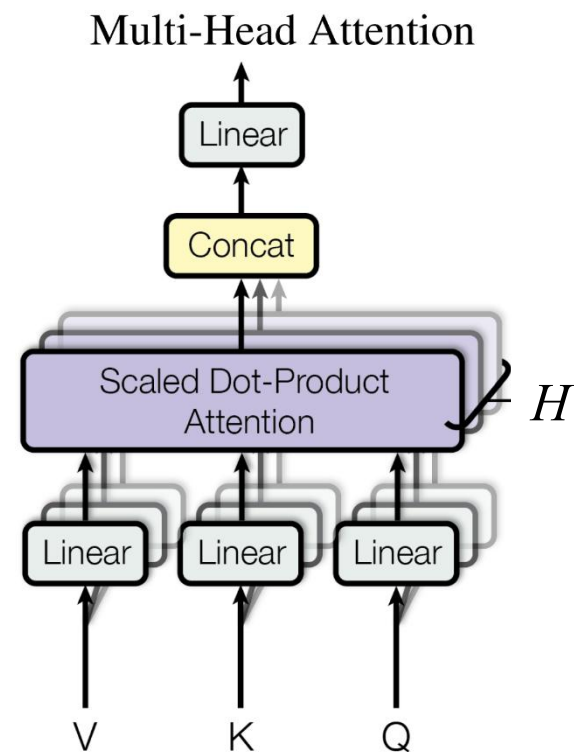
- 加权和

$$\mathbf{x}_t^h = \sum_{i=1}^n \alpha_{ti}^h (\mathbf{W}_h^V \mathbf{x}_i)$$

- 多头特征融合

$$\mathbf{x}_t' = \mathbf{W}^O \text{Concat}(\mathbf{x}_t^1, \mathbf{x}_t^2, \dots, \mathbf{x}_t^H)$$

$\text{Concat}()$: 对向量/矩阵的拼接操作



6.9.7 Transformer

- 多头自注意力的矩阵表示

对于输入序列 $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$, 计算所有数据点的特征表示

记 $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n] \in R^{d \times n}$

- 注意力系数

$$\Lambda^h = \text{softmax}((\mathbf{W}_h^Q \mathbf{X})^T (\mathbf{W}_h^K \mathbf{X})) \in R^{n \times n} \quad \text{非负、列和为1}$$

- 加权和

$$\mathbf{X}^h = (\mathbf{W}_h^V \mathbf{X}) \Lambda^h \quad h = 1, \dots, H$$

- 多头融合

$$\mathbf{X}' = \mathbf{W}^O \text{Concat}(\mathbf{X}^1, \mathbf{X}^2, \dots, \mathbf{X}^H)$$

6.9.7 Transformer

- 自注意力模块：两个子模块
 - 注意力子模块

$$\mathbf{X}' = \text{MultiHeadAttention}(\mathbf{X})$$

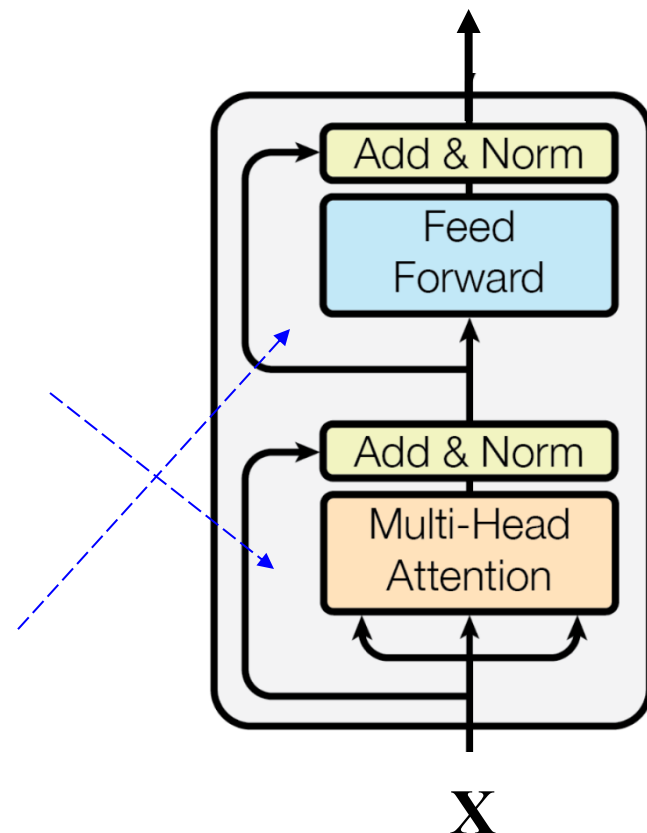
$$\mathbf{X}'' = \text{LayerNorm}(\mathbf{X} + \mathbf{X}')$$

残差连接&
层规范化

- 逐时刻数据特征变换

$$\mathbf{X}''' = \mathbf{W}_2 \text{ReLU}(\mathbf{W}_1 \mathbf{X}'' + \mathbf{b}_1) + \mathbf{b}_2$$

$$\mathbf{X}'''' = \text{LayerNorm}(\mathbf{X}'' + \mathbf{X}''')$$



- ✓ 输入序列与输出序列的长度相同
- ✓ 将多个自注意力模块串联起来，构成Transformer encoder

6.9.7 Transformer

- 很多任务中，存在输入序列与输出序列不等长（如机器翻译）甚至不同模态（如图像描述）的问题
- 可采用基于encoder-decoder结构的模型处理此类问题
 - encoder负责对源数据提取特征
 - decoder负责输出目标数据
- 如何让decoder有效利用encoder的输出？
 - 交叉注意力(cross attention)

6.9.7 Transformer

- 交叉注意力(cross-attention)

- 假设编码器的输出为 $\mathbf{Z} = [\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_n] \in R^{d_1 \times n}$
- 假设解码器第 t 时刻输入为 $\mathbf{y}_t \in R^{d_2}$

相似度:
$$s_{ti} = (\mathbf{W}^Q \mathbf{y}_t)^T (\mathbf{W}^K \mathbf{z}_i)$$

注意力系数:
$$\alpha_{ti} = \frac{\exp(s_{ti})}{\sum_{j=1}^n \exp(s_{tj})}, \quad i = 1, \dots, n$$

加权和:
$$\mathbf{y}_t' = \sum_{i=1}^n \alpha_{ti} (\mathbf{W}^V \mathbf{z}_i)$$

- ✓ 通过注意力访问全部编码器输出数据
- ✓ 可类似使用多头注意力

6.9.7 Transformer

- 交叉注意力模块：三个子模块

设编码器的输出为 $Z = [z_1, z_2, \dots, z_n] \in R^{d_1 \times n}$

解码器当前的输入为 $Y = [y_1, y_2, \dots, y_{t-1}] \in R^{d_2 \times (t-1)}$

- 自注意力子模块

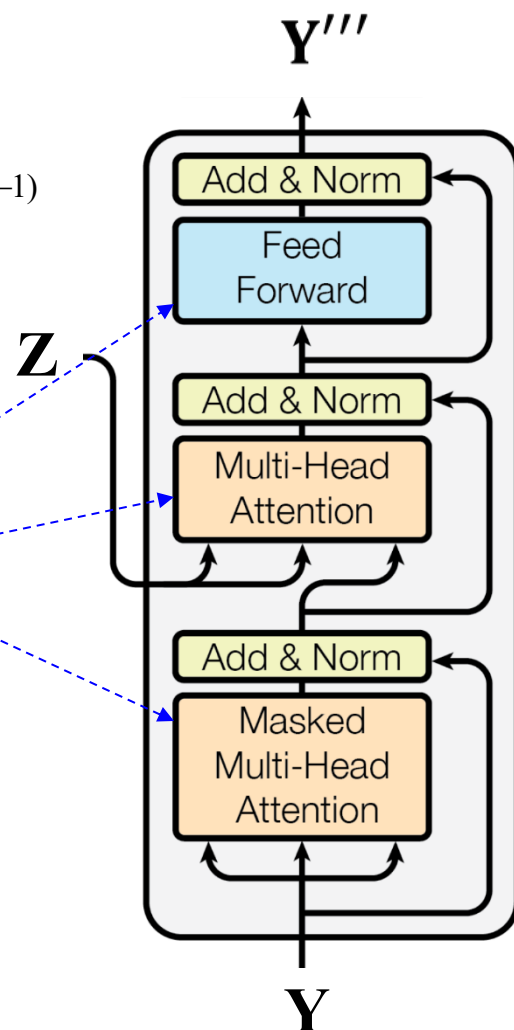
$$Y' = \text{Self-Attention}(Y)$$

- 交叉注意力子模块

$$Y'' = \text{Cross-Attention}(Y', Z)$$

- 逐时刻数据特征变换

$$Y''' = \text{FFN}(Y'')$$



6.9.7 Transformer

- 基于Transformer的机器翻译

$X=\{\text{'hello'}, \text{'world'}, \text{'!'}\} \rightarrow Y=\{\text{'世'}, \text{'界'}, \text{'你'}, \text{'好'}, \text{'!'}\}$

— 编码器

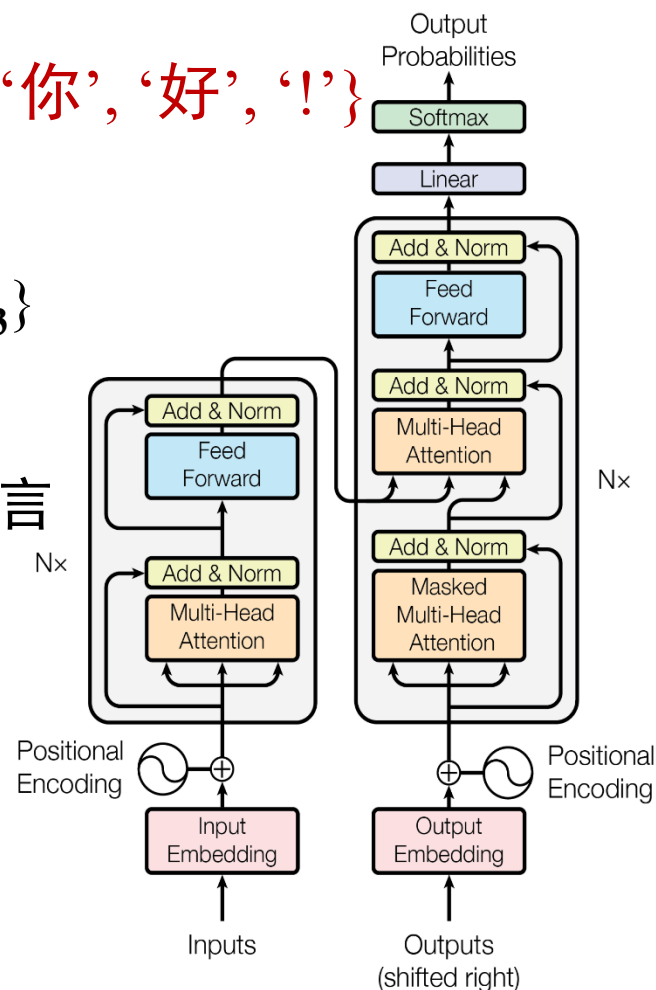
对源语言序列 X 提特征，得到 $Z=\{z_1, z_2, z_3\}$

— 解码器

以自回归(autoregressive)方式输出目标语言

第 t 时刻，根据已翻译出的结果 $\{y_1, y_2, \dots, y_{t-1}\}$ 和 Z 预测 y_t

$$P(y_t | y_1, \dots, y_{t-1}, Z)$$



6.9.7 Transformer

- Transformer和Attention机制本质上是顺序无关的，如何表示位置？

- 位置编码

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos}/10000^{2i/d_{\text{model}}})$$

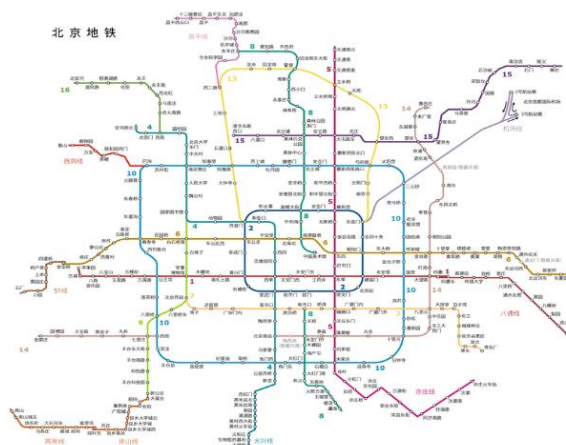
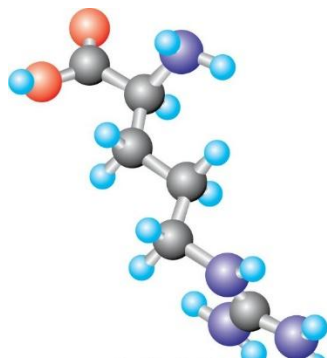
$$\text{PE}(\text{pos}, 2i+1) = \cos(\text{pos}/10000^{2i/d_{\text{model}}})$$

- 如何表示字/词/token？

- 分布式词表示(distributed word representation)

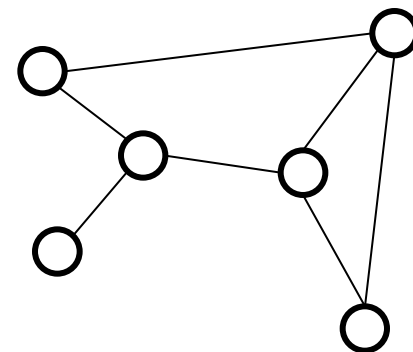
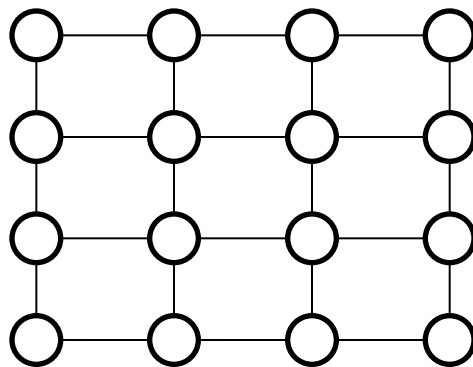
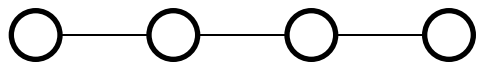
6.9.8 图神经网络 (Graph Neural Networks)

- 图(graph)是一类广泛存在的数据形式
 - 自然界中的图
 - 分子、蛋白质、生物网络
 - 人类社会中的图
 - 传感器网络、交通网络、能源网络
 - 虚拟世界中的图
 - 社交网络、引用网络、知识图谱



6.9.8 图神经网络 (Graph Neural Networks)

- 数学上，图是一种抽象的数据结构
 - $G = (V, E)$: 图是节点和边的集合
 - $V = \{v_i\}$: 节点描述实体，如：人、站点、原子等
 - $E = \{e_{ij}\}$: 边描述实体间关系，如：交往、购买、距离、作用力等



sequence: 如文本、语音

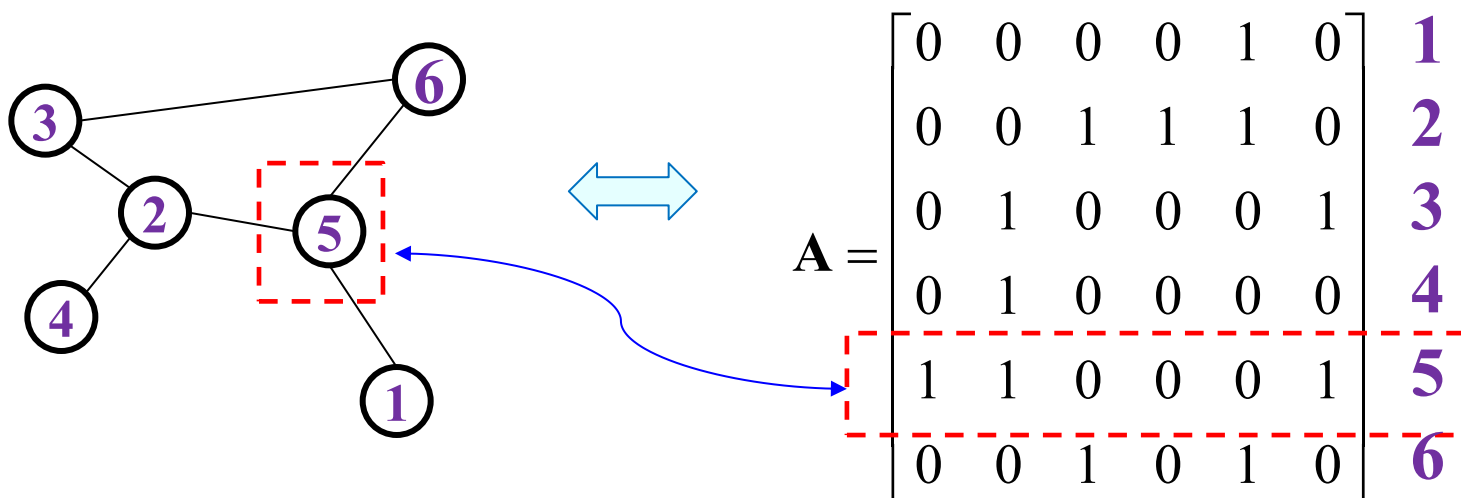
grid: 如图像、视频

general graph: 任意大小和拓扑结构

6.9.8 图神经网络 (Graph Neural Networks)

- 数学上，图是一种抽象的数据结构
 - $G = (V, E)$: 图是节点和边的集合
 - $V = \{v_i\}$: 节点描述实体，如：人、地点、原子等
 - $E = \{e_{ij}\}$: 边描述实体间关系，如：交往、购买、距离、作用力等

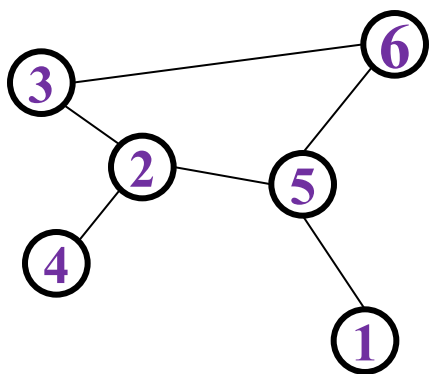
邻接矩阵表示



如果 v_i, v_j 相连, $A_{ij}=1$; 否则, $A_{ij}=0$

6.9.8 图神经网络 (Graph Neural Networks)

- 模式识别/机器学习中的图
 - 除了图结构 $G=(V, E)$ ，节点和边通常还具有特征，一般由向量表示



节点	特征1	特征2	...
v_1	-1	0	
v_2	0	0	
v_3			
v_4			
v_5			
v_6			

边	特征1	特征2	...
e_{15}	8	-0.1	
e_{23}	2	0	
e_{24}			
e_{25}			
e_{36}			
e_{56}			



$$\mathbf{A} \in \{0,1\}^{n \times n}$$



$$\mathbf{X} = \{\mathbf{x}_i \in \mathbf{R}^{d_1}, v_i \in V\}$$



$$\mathbf{Y} = \{\mathbf{y}_{ij} \in \mathbf{R}^{d_2}, e_{ij} \in E\}$$

6.9.8 图神经网络 (Graph Neural Networks)

- 图的分类
 - 有向图vs无向图、静态图vs动态图、同质图vs异质图、同配图vs异配图
- 图上的学习任务
 - Graph level
 - 图分类/图属性预测：该分子具有某种性质吗？
 - 图生成：生成具有某种性质的分子结构
 - 图匹配：这两个蛋白质有多相似？
 - Node level
 - 节点分类/节点属性预测：该地铁站在 t 时刻的流量？
 - 节点聚类/社区发现：哪些人会被该病毒感染？
 - Edge level
 - 边分类/边属性预测：用户a会关注用户b吗？

6.9.8 图神经网络 (Graph Neural Networks)

- 如何设计适合图数据的模型？
 - 同时利用图的拓扑结构、节点特征、边特征学习节点表示和边表示
- 图神经网络的基本思想
 - 以“边”为基础进行近邻节点间的信息传递
 - 节点利用从近邻收到的信息更新自己的表示

6.9.8 图神经网络 (Graph Neural Networks)

- 一个简单的例子

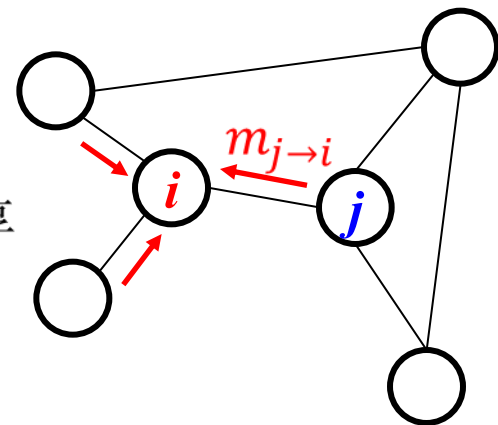
- 消息传递

$$\mathbf{m}_{j \rightarrow i} = \mathbf{W} \mathbf{h}_j$$

$\mathbf{h}_j \in \mathbb{R}^d$: 节点 j 的特征表示

$\mathbf{W} \in \mathbb{R}^{d \times d'}$: 可学习参数矩阵, 所有节点共享

$\mathbf{m}_{j \rightarrow i} \in \mathbb{R}^{d'}$: 节点 j 传递给 i 的信息



- 消息汇聚

$$\mathbf{m}_i = \frac{1}{d_i} \sum_{j \in N(i)} \mathbf{m}_{j \rightarrow i}$$

d_i : 节点 i 直接相连的节点个数

$N(i)$: 所有与节点 i 直接相连的节点集合

$\mathbf{m}_i \in \mathbb{R}^{d'}$: 节点 i 接收到的全部消息

- 节点表示更新

$$\mathbf{h}_i' = \sigma(\mathbf{W} \mathbf{h}_i + \mathbf{m}_i)$$

$\mathbf{h}_i' \in \mathbb{R}^d$: 更新后节点 i 的特征表示

6.9.8 图神经网络 (Graph Neural Networks)

- 上述过程的矩阵表示

$$\mathbf{H}' = \sigma((\mathbf{I} + \mathbf{D}^{-1}\mathbf{A})\mathbf{H}\mathbf{W})$$

$\mathbf{A} \in \{0,1\}^{n \times n}$: 邻接矩阵

$\mathbf{D} \in R^{n \times n}$: 对角矩阵, $\mathbf{D}_{ii}=d_i$, 其他元素为零

$\mathbf{H} \in R^{n \times d}$: 节点的初始特征表示

$\mathbf{W} \in R^{d \times d'}$: 可学习参数矩阵

$\mathbf{H}' \in R^{n \times d'}$: 更新后的节点特征表示

- 两层图网络

$$\mathbf{H}' = \sigma((\mathbf{I} + \mathbf{D}^{-1}\mathbf{A})\mathbf{H}\mathbf{W}_1)$$

$$\mathbf{H}'' = \sigma((\mathbf{I} + \mathbf{D}^{-1}\mathbf{A})\mathbf{H}'\mathbf{W}_2)$$

6.9.8 图神经网络 (Graph Neural Networks)

- 基于消息传递机制的图神经网络（一般形式）

- 消息传递 $m_{j \rightarrow i} = f_m(h_i, h_j, h_{j \rightarrow i})$
- 消息汇聚 $m_i = \text{Aggr}(\{m_{j \rightarrow i}, j \in N(i)\})$
- 节点表示更新 $h_i' = f_{\text{node}}(h_i, m_i)$
- 边表示更新 $h_{j \rightarrow i}' = f_{\text{edge}}(h_i, m_i, h_j, m_j, h_{j \rightarrow i})$

h_i : 节点 i 的特征表示

$h_{j \rightarrow i}$: 边 e_{ij} 的特征表示

$f_m, \text{Aggr}, f_{\text{node}}, f_{\text{edge}}$: 被所有节点共享的函数

Aggr : 与顺序无关(order invariant), 如求平均

$f_m, f_{\text{node}}, f_{\text{edge}}$: 包含可学习参数, 如MLP

6.9.8 图神经网络 (Graph Neural Networks)

- 图神经网络对比卷积神经网络
 - 如果将节点看作像素，图上的消息传递操作和卷积操作非常相似：局部连接、权值共享
 - 因此，图神经网络也被称为图卷积网络(Graph Convolution Networks, GCN)

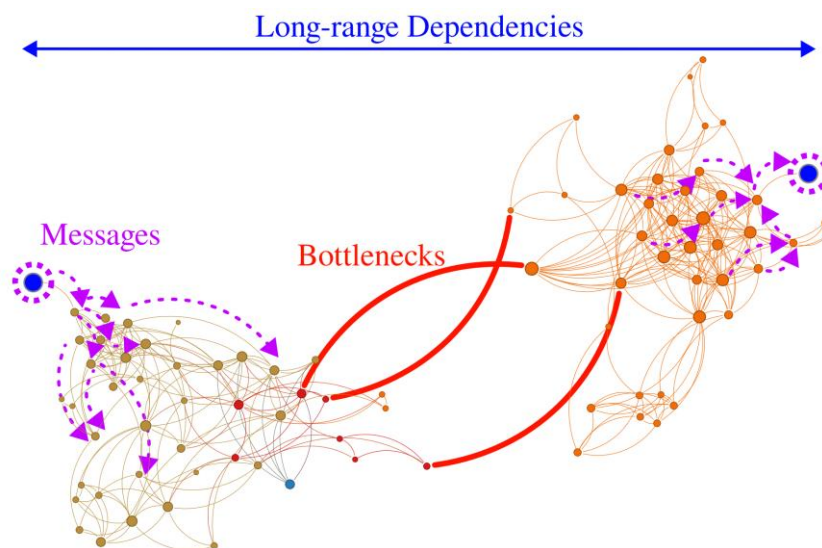
6.9.8 图神经网络 (Graph Neural Networks)

- 图神经网络的问题
 - 过平滑(over-smoothing)

$$h'_i = \sigma\left(\sum_{j \in N(i) \cup \{i\}} \frac{1}{c_{ij}} W h_j\right)$$

Normalization factor

- 过挤压(over-squashing)



6.9.8 图神经网络 (Graph Neural Networks)

- GNN上的预测

- Node level

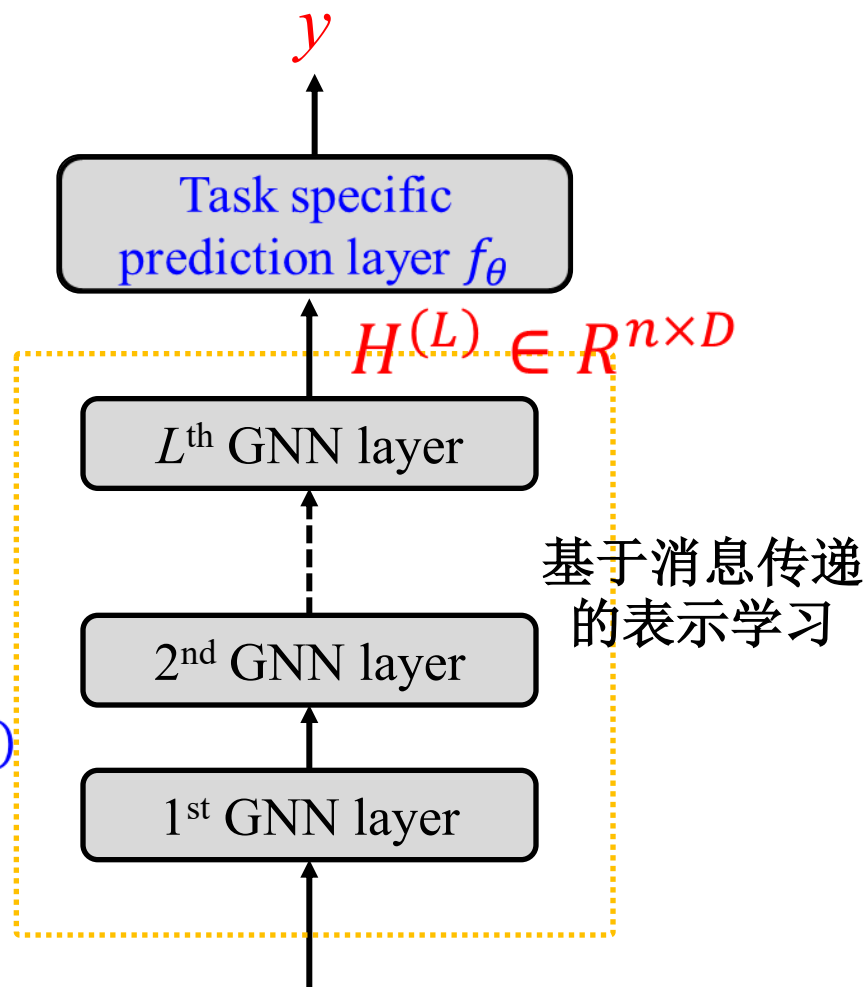
- $y_i = f_{\theta}(h_i^{(L)})$

- Edge level

- $y_{ij} = f_{\theta}(h_i^{(L)}, h_j^{(L)})$

- Graph level

- $y_G = f_{\theta}(\text{pooling}(\{h_i^{(L)}, i = 1, \dots, n\}))$



$$(A \in \{0,1\}^{n \times n}, X \in R^{n \times d})$$

n : 节点数, d, D : 特征维度 第115页

致谢

- PPT由向世明老师提供

Thank All of You!
(Questions?)

张燕明

ymzhang@nlpr.ia.ac.cn

people.ucas.ac.cn/~ymzhang

模式分析与学习课题组 (PAL)

中科院自动化研究所· 多模态人工智能系统实验室